

---

**UNIT 4****IMPLEMENTATION TECHNIQUES****SYLLABUS**

RAID - File Organization – Organization of Records in Files – Indexing and Hashing –Ordered Indices – B+ tree Index Files – B tree Index Files – Static Hashing – Dynamic Hashing – Query Processing Overview – Algorithms for SELECT and JOIN operations – Query optimization using Heuristics and Cost Estimation.

**4.1 RAID**

- The general idea behind RAID is to employ a group of hard drives together with some form of duplication, either to increase reliability or to speed up operations, (or sometimes both)
- RAID originally stood for Redundant Array of Independent Disks, and was designed to use a bunch of cheap small disks in place of one or two larger more expensive ones.
- RAID systems employ large possibly expensive disks as their components, switching the definition to Independent disks.

**4.1.1 Improvement of Reliability via Redundancy**

The more disks a system has, the greater the likelihood that one of them will go bad at any given time. Hence increasing disks on a system actually decreases the Mean Time To Failure, MTTF of the system.

- If, however, the same data was copied onto multiple disks, then the data would not be lost unless both (or all) copies of the data were damaged simultaneously, which a much lower probability than for a single disk is going bad. More specifically, the second disk would have to go bad before the first disk was repaired, which brings the Mean Time To Repair into play.
- This is the basic idea behind disk mirroring, in which a system contains identical data on two or more disks.
  - ✓ Note that a power failure during a write operation could cause both disks to contain corrupt data, if both disks were writing simultaneously at the time of the power failure. One solution is to write to the two disks in series, so that they will not both become corrupted (at least not in the same way) by a power failure.

---

#### 4.1.2 Improvement of Performance via Parallelism

- ✓ There is also a performance benefit to mirroring, particularly with respect to reads. Since every block of data is duplicated on multiple disks, read operations can be satisfied from any available copy, and multiple disks can be reading different data blocks simultaneously in parallel.
- ✓ Another way of improving disk access time is with striping, which basically means spreading data out across multiple disks that can be accessed simultaneously.
  - With bit-level striping the bits of each byte are striped across multiple disks. For example if 8 disks were involved, then each 8-bit byte would be read in parallel by 8 heads on separate disks. A single disk read would access  $8 * 512$  bytes = 4K worth of data in the time normally required to read 512 bytes. Similarly if 4 disks were involved, then two bits of each byte could be stored on each disk, for 2K worth of disk access per read or write operation.
  - Block-level striping spreads a file system across multiple disks on a block-by-block basis, so if block N were located on disk 0, then block N + 1 would be on disk 1, and so on. This is particularly useful when file systems are accessed in clusters of physical blocks. Other striping possibilities exist, with block-level striping being the most common.

#### 4.1.3 RAID Levels

Mirroring provides reliability but is expensive; Striping improves performance, but does not improve reliability. Accordingly there are a number of different schemes that combine the principals of mirroring and striping in different ways, in order to balance reliability versus performance versus cost. These are described by different RAID levels, as follows:

- **Raid Level 0** - This level includes striping only, with no mirroring.
- **Raid Level 1** - This level includes mirroring only, no striping.
- **Raid Level 2** - This level stores error-correcting codes on additional disks, allowing for any damaged data to be reconstructed by subtraction from the remaining undamaged data.

- **Raid Level 3** - This level is similar to level 2, except that it takes advantage of the fact that each disk is still doing its own error-detection, so that when an error occurs, there is no question about which disk in the array has the bad data.
- **Raid Level 4** - This level is similar to level 3, employing block-level striping instead of bit-level striping. The benefits are that multiple blocks can be read independently, and changes to a block only require writing two blocks (data and parity) rather than involving all disks.
- **Raid Level 5** - This level is similar to level 4, except the parity blocks are distributed over all disks, thereby more evenly balancing the load on the system.
- **Raid Level 6** - This level extends raid level 5 by storing multiple bits of error-recovery codes, for each bit position of data, rather than a single parity bit. In the example shown below 2 bits of ECC are stored for every 4 bits of data, allowing data recovery in the face of up to two simultaneous disk failures.

There are also two RAID levels which combine RAID levels 0 and 1 (striping and mirroring) in different combinations, designed to provide both performance and reliability at the expense of increased cost.

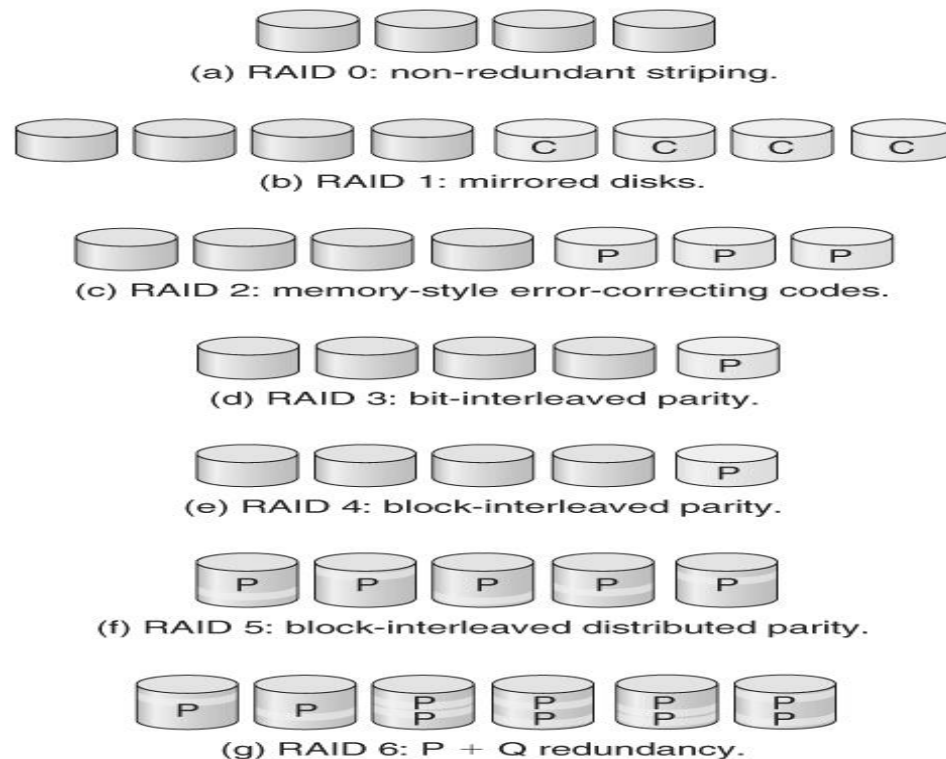


Fig 4.1: RAID levels.

- **RAID level 0 + 1** disks are first striped, and then the striped disks mirrored to another set. This level generally provides better performance than RAID level 5.
- **RAID level 1 + 0** mirrors disks in pairs, and then stripes the mirrored pairs. The storage capacity, performance, etc. are all the same, but there is an advantage to this approach in the event of multiple disk failures,

#### 4.1.4 Selecting a RAID Level

- Trade-offs in selecting the optimal RAID level for a particular application include cost, volume of data, need for reliability, need for performance, and rebuild time, the latter of which can affect the likelihood that a second disk will fail while the first failed disk is being rebuilt.
- Other decisions include how many disks are involved in a RAID set and how many disks to protect with a single parity bit. More disks in the set increases performance but increases cost. Protecting more disks per parity bit saves cost, but increases the likelihood that a second disk will fail before the first bad disk is repaired.

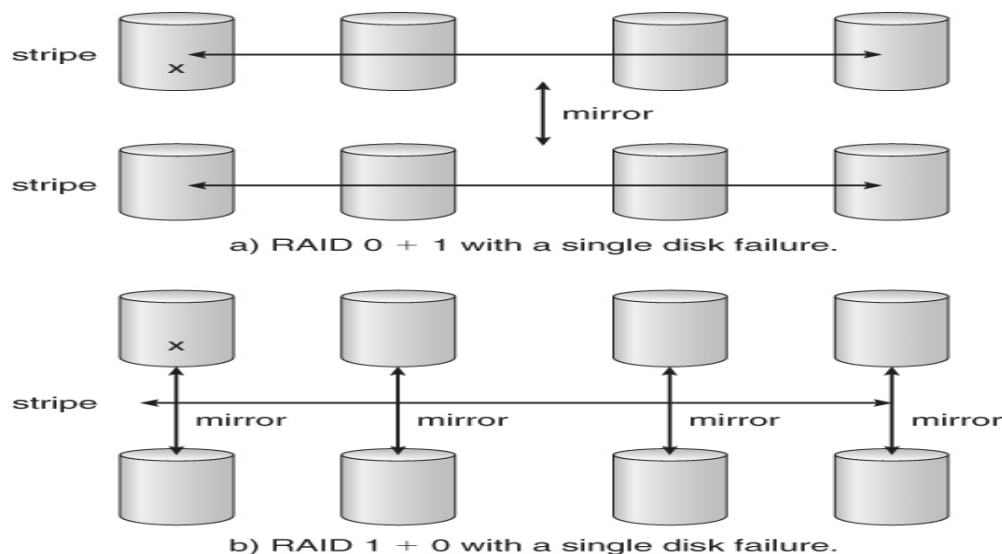
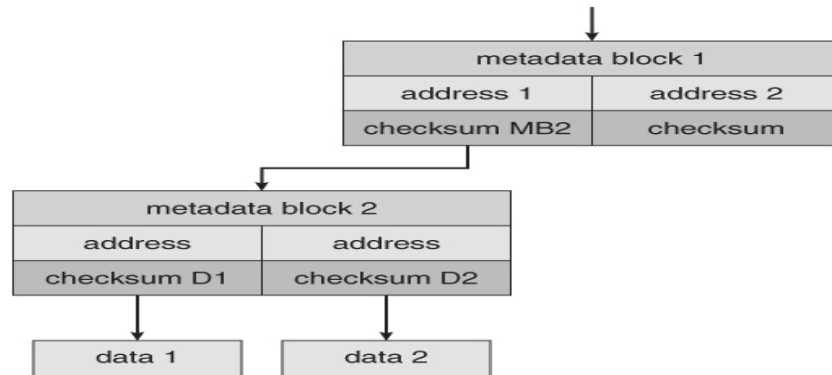


Fig 4.2: RAID 0 + 1 and 1 + 0

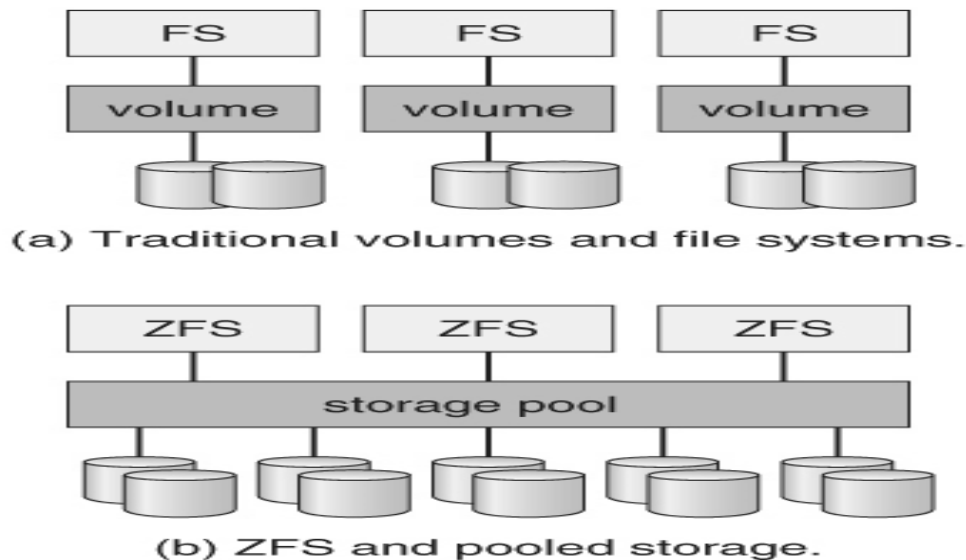
#### 4.1.5 Problems with RAID

- RAID protects against physical errors, but not against any number of bugs or other errors that could write erroneous data.



**Fig 4.3: ZFS checksums all metadata and data**

- ZFS adds an extra level of protection by including data block checksums in all i nodes along with the pointers to the data blocks. If data are mirrored and one copy has the correct checksum and the other does not, then the data with the bad checksum will be replaced with a copy of the data with the good checksum.
- Another problem with traditional file systems is that the sizes are fixed, and relatively difficult to change. Where RAID sets are involved it becomes even harder to adjust file system sizes, because a file system cannot span across multiple file systems.



**Fig 4.4: (a) Traditional volumes and file systems. (b) ZFS pool and file systems.**

- ZFS solves these problems by pooling RAID sets, and by dynamically allocating space to file systems as needed. File system sizes can be limited by quotas, and space can also be reserved to guarantee that a file system will be able to grow later, but these

---

parameters can be changed at any time by the file system's owner. Otherwise file systems grow and shrink dynamically as needed.

## 4.2 FILE ORGANIZATION

A database consists of a huge amount of data. The data is grouped within a table in RDBMS, and each table has related records. A user can see that the data is stored in form of tables, but in actual this huge amount of data is stored in physical memory in form of files.

**File** – A file is named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks.

### What is File Organization?

File Organization refers to the logical relationships among various records that constitute the file, particularly with respect to the means of identification and access to any specific record. In simple terms, Storing the files in certain order is called file Organization. **File Structure** refers to the format of the label and data blocks and of any logical control record.

#### 4.2.1 Files, Fixed-Length Records, and Variable-Length Records

A file is a sequence of records. In many cases, all records in a file are of the same record type. If every record in the file has exactly the same size (in bytes), the file is said to be made up of fixed-length records.

If different records in the file have different sizes, the file is said to be made up of variable-length records. A file may have variable-length records for several reasons:

- The file records are of the same record type, but one or more of the fields are of varying size (variable-length fields). For example, the NAME field of EMPLOYEE can be a variable-length field.

- The file records are of the same record type, but one or more of the fields may have multiple values for individual records; such a field is called a repeating field and a group of values for the field is often called a repeating group.
- The file records are of the same record type, but one or more of the fields are optional; that is, they may have values for some but not the entire file records (optional fields).

The file contains records of different record types and hence of varying size (mixed file). This would occur if related records of different types were clustered (placed together) on disk blocks; for example, the GRADE\_REPORT records of a particular student may be placed following that STUDENT's record.

#### **4.2.2 Types of File Organizations –**

Various methods have been introduced to Organize files. These particular methods have advantages and disadvantages on the basis of access or selection. Thus it is all upon the programmer to decide the best suited file Organization method according to his requirements.

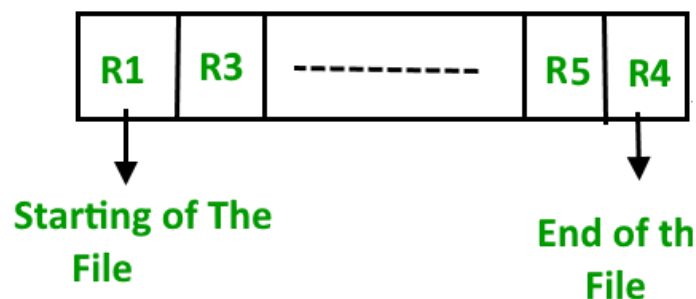
##### **Some types of File Organizations are:**

- Sequential File Organization
- Heap File Organization
- Hash File Organization
- B+ Tree File Organization
- Clustered File Organization

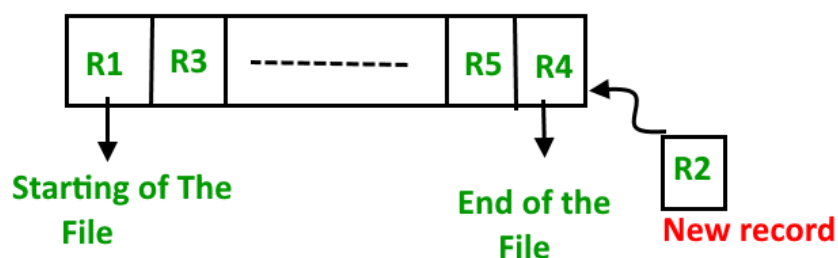
#### 4.2.2.1 Sequential File Organization –

The easiest method for file Organization is Sequential method. In this method the file are stored one after another in a sequential manner. There are two ways to implement this method:

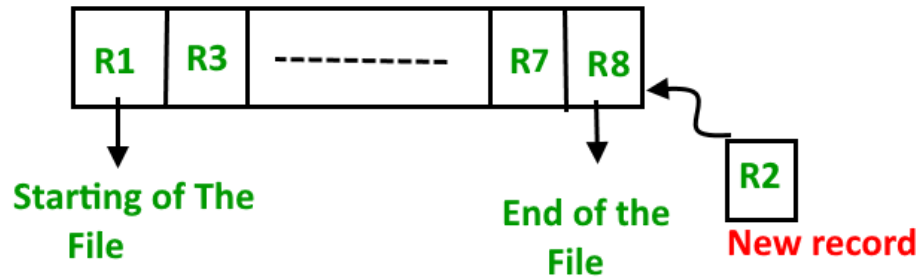
1. **Pile File Method** – This method is quite simple, in which we store the records in a sequence i.e one after other in the order in which they are inserted into the tables.



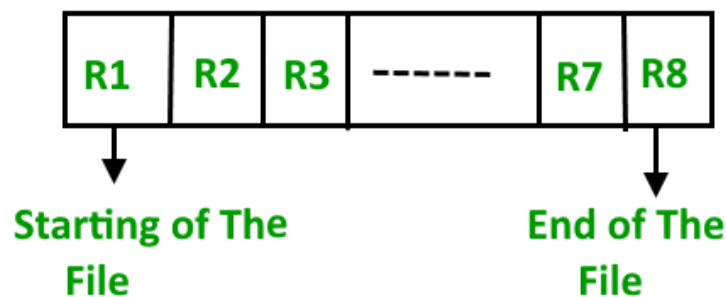
Insertion of new record – Let the R1, R3 and so on upto R5 and R4 be four records in the sequence. Here, records are nothing but a row in any table. Suppose a new record R2 has to be inserted in the sequence, and then it is simply placed at the end of the file.



2. **Sorted File Method** – In this method, as the name itself suggest whenever a new record has to be inserted, it is always inserted in a sorted (ascending or descending) manner. Sorting of records may be based on any primary key or any other key.



**Insertion of new record** – Let we assume that there is a preexisting sorted sequence of four records R1, R3, and so on upto R7 and R8. Suppose a new record R2 has to be inserted in the sequence, then it will be inserted at the end of the file and then it will sort the sequence.



### Pros and Cons of Sequential File Organization –

#### Pros –

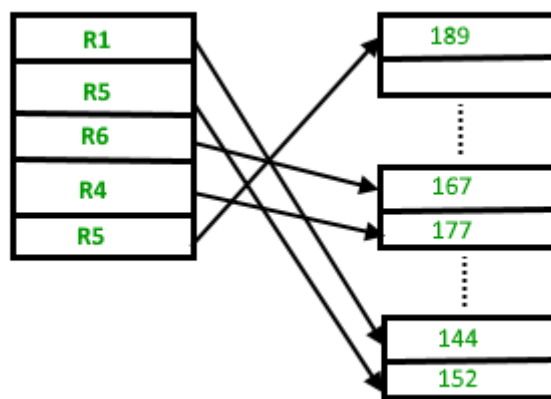
- Fast and efficient method for huge amount of data.
- Simple design.
- Files can be easily stored in magnetic tapes i.e cheaper storage mechanism.

#### Cons –

- Time wastage as we cannot jump on a particular record that is required, but we have to move in a sequential manner which takes our time.
- Sorted file method is inefficient as it takes time and space for sorting records.

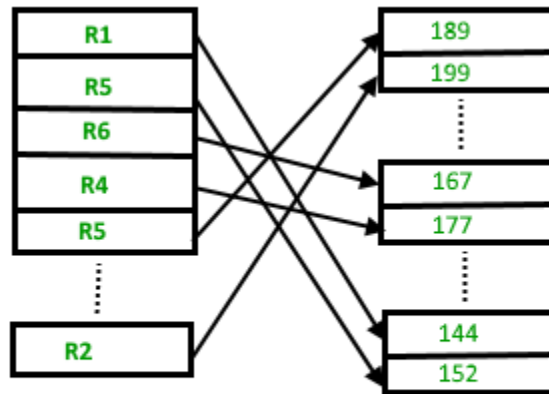
#### 4.2.2.2 Heap File Organization –

Heap File Organization works with data blocks. In this method records are inserted at the end of the file, into the data blocks. No Sorting or Ordering is required in this method. If a data block is full, the new record is stored in some other block, Here the other data block need not be the very next data block, but it can be any block in the memory. It is the responsibility of DBMS to store and manage the new records.



##### Insertion of new record –

Suppose we have four records in the heap R1, R5, R6, R4 and R3 and suppose a new record R2 has to be inserted in the heap then, since the last data block i.e data block 3 is full it will be inserted in any of the data blocks selected by the DBMS, lets say data block 1.



If we want to search, delete or update data in heap file Organization then we will traverse the data from the beginning of the file till we get the requested record. Thus if the database is very huge, searching, deleting or updating the record will take a lot of time.

#### **Pros and Cons of Heap File Organization –**

##### **Pros –**

- Fetching and retrieving records is faster than sequential record but only in case of small databases.
- When there is a huge number of data needs to be loaded into the database at a time, then this method of file Organization is best suited.

##### **Cons –**

- Problem of unused memory blocks.
- Inefficient for larger databases.

#### **4.2.2.3 HASHING FILE ORGANIZATION**

In database management system, when we want to retrieve a particular data, It becomes very inefficient to search all the index values and reach the desired data. In this situation, Hashing technique comes into picture.

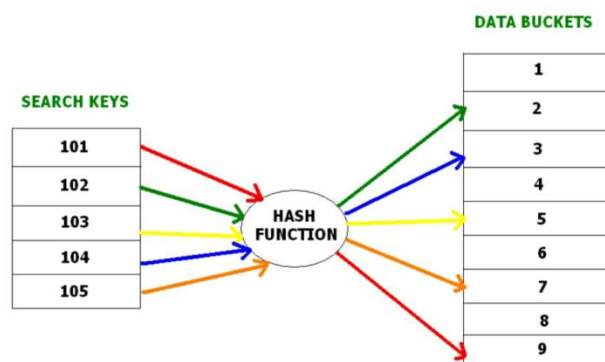
Hashing is an efficient technique to directly search the location of desired data on the disk

without using index structure. Data is stored at the data blocks whose address is generated by using hash function. The memory location where these records are stored is called as data block or data bucket.

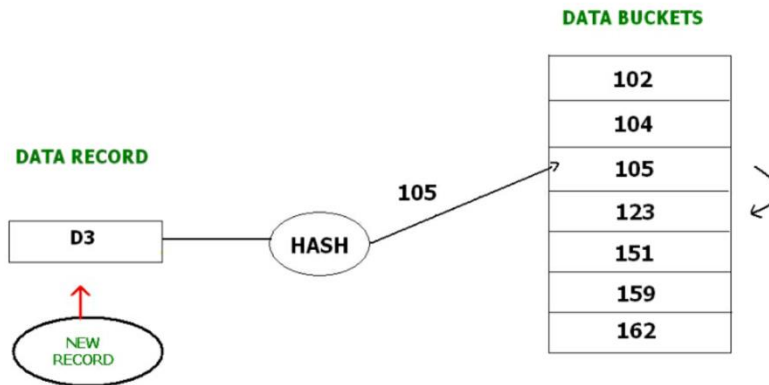
### Hash File Organization:

- **Data bucket** – Data buckets are the memory locations where the records are stored. These buckets are also considered as *Unit Of Storage*.
- **Hash Function** – Hash function is a mapping function that maps all the set of search keys to actual record address. Generally, hash function uses primary key to generate the hash index – address of the data block. Hash function can be simple mathematical function to any complex mathematical function.
- **Hash Index**-The prefix of an entire hash value is taken as a hash index. Every hash index has a depth value to signify how many bits are used for computing a hash function. These bits can address  $2^n$  buckets. When all these bits are consumed ? then the depth value is increased linearly and twice the buckets are allocated.

Below given diagram clearly depicts how hash function work:



Hashing is further divided into two sub categories:



### Static Hashing –

In static hashing, when a search-key value is provided, the hash function always computes the same address. For example, if we want to generate address for STUDENT\_ID = 76 using mod (5) hash function, it always result in the same bucket address 4. There will not be any changes to the bucket address here. Hence number of data buckets in the memory for this static hashing remains constant throughout.

### Operations –

- **Insertion** – When a new record is inserted into the table, The hash function  $h$  generate a bucket address for the new record based on its hash key  $K$ .  
Bucket address =  $h(K)$
- **Searching** – When a record needs to be searched, The same hash function is used to retrieve the bucket address for the record. For Example, if we want to retrieve whole record for ID 76, and if the hash function is mod (5) on that ID, the bucket address generated would be 4. Then we will directly got to address 4 and retrieve the whole record for ID 104. Here ID acts as a hash key.
- **Deletion** – If we want to delete a record, Using the hash function we will first fetch the record which is supposed to be deleted. Then we will remove the records for that address in memory.

- **Updation** – The data record that needs to be updated is first searched using hash function, and then the data record is updated.

Now, If we want to insert some new records into the file But the data bucket address generated by the hash function is not empty or the data already exists in that address. This becomes a critical situation to handle. This situation in the static hashing is called **bucket overflow**.

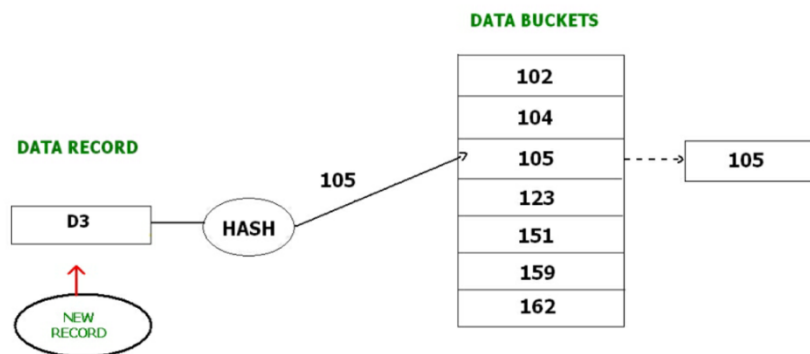
How will we insert data in this case?

There are several methods provided to overcome this situation. Some commonly used methods are discussed below:

### 1. Open Hashing –

In Open hashing method, next available data block is used to enter the new record, instead of overwriting older one. This method is also called linear probing.

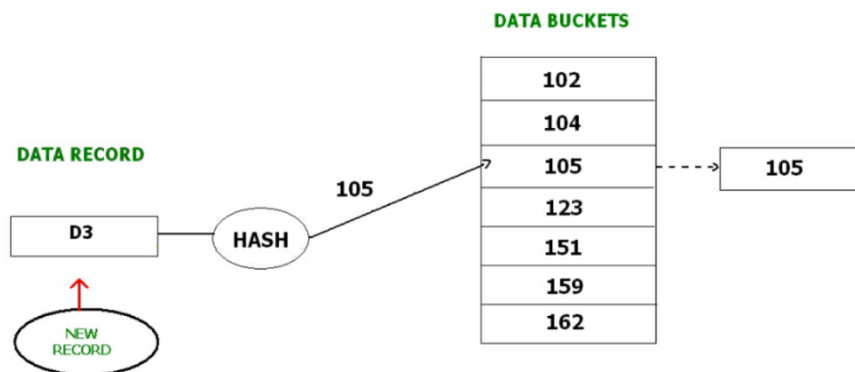
For example, D3 is a new record which needs to be inserted , the hash function generates address as 105. But it is already full. So the system searches next available data bucket, 123 and assigns D3 to it.



### Closed hashing –

In Closed hashing method, a new data bucket is allocated with same address and is linked it after the full data bucket. This method is also known as overflow chaining.

For example, we have to insert a new record D3 into the tables. The static hash function generates the data bucket address as 105. But this bucket is full to store the new data. In this case a new data bucket is added at the end of 105 data bucket and is linked to it. Then new record D3 is inserted into the new bucket.



- **Quadratic probing:**

Quadratic probing is very much similar to open hashing or linear probing. Here, The only difference between old and new bucket is linear. Quadratic function is used to determine the new bucket address.

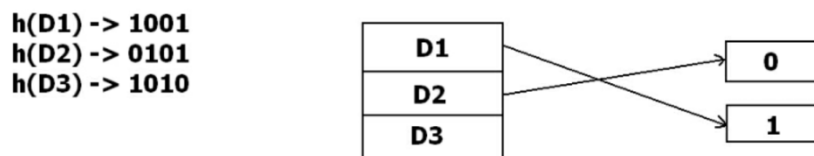
- **Double Hashing:**

Double Hashing is another method similar to linear probing. Here the difference is fixed as in linear probing, but this fixed difference is calculated by using another hash function. That's why the name is double hashing.

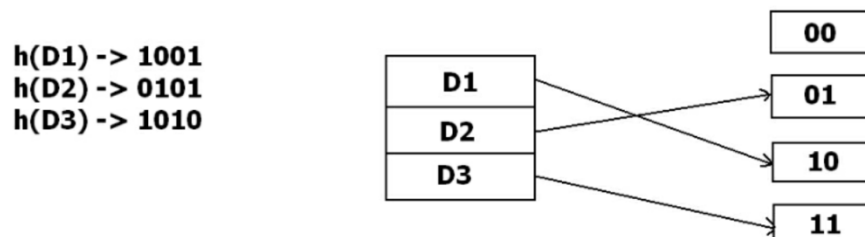
### **Dynamic Hashing –**

The drawback of static hashing is that it does not expand or shrink dynamically as the size of the database grows or shrinks. In Dynamic hashing, data buckets grows or shrinks (added or removed dynamically) as the records increases or decreases. Dynamic hashing is also known as extended hashing.

In dynamic hashing, the hash function is made to produce a large number of values. For Example, there are three data records D1, D2 and D3 . The hash function generates three addresses 1001, 0101 and 1010 respectively. This method of storing considers only part of this address – especially only first one bit to store the data. So it tries to load three of them at address 0 and 1.



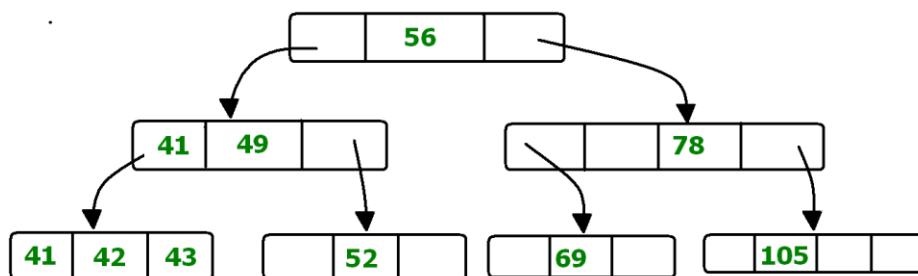
But the problem is that No bucket address is remaining for D3. The bucket has to grow dynamically to accommodate D3. So it changes the address have 2 bits rather than 1 bit, and then it updates the existing data to have 2 bit address. Then it tries to accommodate D3.



#### 4.2.2.3 B+ Tree File Organization –

B+ Tree, as the name suggests, It uses a tree like structure to store records in File. It uses the concept of Key indexing where the primary key is used to sort the records. For each primary key, an index value is generated and mapped with the record. An index of a record is the address of record in the file.

B+ Tree is very much similar to binary search tree, with the only difference that instead of just two children, it can have more than two. All the information is stored in leaf node and the intermediate nodes acts as pointer to the leaf nodes. The information in leaf nodes always remain a sorted sequential linked list.



In the above diagram 56 is the root node which is also called the main node of the tree.

The intermediate nodes here, just consist the address of leaf nodes. They do not contain any actual record. Leaf nodes consist of the actual record. All leaf nodes are balanced.

#### Pros and Cons of B+ Tree File Organization –

##### Pros –

- Tree traversal is easier and faster.

- Searching becomes easy as all records are stored only in leaf nodes and are sorted sequential linked list.
- There is no restriction on B+ tree size. It may grows/shrink as the size of data increases/decreases.

#### Cons –

- Inefficient for static tables.

#### 4.2.2.4 Cluster File Organization –

In cluster file organization, two or more related tables/records are stored within same file known as clusters. These files will have two or more tables in the same data block and the key attributes which are used to map these tables together are stored only once.

Thus it lowers the cost of searching and retrieving various records in different files as they are now combined and kept in a single cluster.

For example we have two tables or relation Employee and Department. These tables are related to each other.

| EMPLOYEE |          |           |        | DEPARTMENT |          |
|----------|----------|-----------|--------|------------|----------|
| EMP ID   | EMP_NAME | EMP_ADD   | DEP_ID | DEP_ID     | DEP_NAME |
| 01       | JOE      | CAPE TOWN | D_101  | D_101      | ECO      |
| 02       | ANNIE    | FRANSISCO | D_103  | D_102      | CS       |
| 03       | PETER    | CROY CITY | D_101  | D_103      | JAVA     |
| 04       | JOHN     | FRANSISCO | D_102  | D_104      | MATHS    |
| 05       | LUNA     | TOKYO     | D_106  | D_105      | BIO      |
| 06       | SONI     | W.LAND    | D_105  | D_106      | CIVIL    |
| 07       | SAKACHI  | TOKYO     | D_104  |            |          |
| 08       | MARY     | NOVI      | D_101  |            |          |

Therefore these tables are allowed to combine using a join operation and can be seen in a cluster file.

**CLUSTER KEY**


| DEP_ID | DEP_NAME | EMP ID | EMP_NAME | EMP_ADD   |
|--------|----------|--------|----------|-----------|
| D_101  | ECO      | 01     | JOE      | CAPE TOWN |
|        |          | 02     | PETER    | CROY CITY |
|        |          | 03     | MARY     | NOVI      |
| D_102  | CS       | 04     | JOHN     | FRANSISCO |
| D_103  | JAVA     | 05     | ANNIE    | FRANSISCO |
| D_104  | MATHS    | 06     | SAKACHI  | TOKYO     |
| D_105  | BIO      | 07     | SONI     | W.LAND    |
| D_106  | CIVIL    | 08     | LUNA     | TOKYO     |

**DEPARTMENT + EMPLOYEE**

If we have to insert, update or delete any record we can directly do so. Data is sorted based on the primary key or the key with which searching is done. **Cluster key** is the key with which joining of the table is performed.

**Types of Cluster File Organization** – There are two ways to implement this method:

1. **Indexed Clusters** – In Indexed clustering the records are group based on the cluster key and stored together. The above mentioned example of Employee and Department relationship is an example of Indexed Cluster where the records are based on the Department ID.
2. **Hash Clusters** – This is very much similar to indexed cluster with only difference that instead of storing the records based on cluster key, we generate hash key value and store the records with same hash key value.

## 4.4 INDEXING AND HASHING

### 4.4.1 Indexing

The purpose of storing an index is to optimize speed and performance in finding relevant documents for a search query. Without an index, the search engine would scan every document in the corpus, which would require considerable time and computing power

#### 4.4.1.1 Factors

**(a) Index design factors:**

Major factors in designing a search engine's architecture include:

**Merge factors**

- How data enters the index, or how words or subject features are added to the index during text corpus traversal, and whether multiple indexers can work asynchronously.
- The indexer must first check whether it is updating old content or adding new content. Traversal typically correlates to the data collection policy.
- Search engine index merging is similar in concept to the SQL Merge command and other merge algorithms.

**Storage techniques**

How to store the index data, that is, whether information should be data compressed or filtered.

**Index size**

How much computer storage is required to support the index.

**Lookup speed**

How quickly a word can be found in the inverted index. The speed of finding an entry in a data structure, compared with how quickly it can be updated or removed, is a central focus of computer science.

**Maintenance**

How the index is maintained over time.

**Fault tolerance**

How important it is for the service to be reliable. Issues include dealing with index corruption, determining whether bad data can be treated in isolation, dealing with bad hardware, partitioning, and schemes such as hash-based or composite partitioning, as well as replication.

**(b) Index data structures:**

Search engine architectures vary in the way indexing is performed and in methods of index storage to meet the various design factors.

**Suffix tree**

- Figuratively structured like a tree, supports linear time lookup. Built by storing the suffixes of words.
- The suffix tree is a type of tire. Tries support extendable hashing, which is important for search engine indexing.
- Used for searching for patterns in DNA sequences and clustering. A major drawback is that storing a word in the tree may require space beyond that required to store the word itself.

### **Inverted index**

Stores a list of occurrences of each atomic search criterion,<sup>[9]</sup> typically in the form of a hash table or binary tree.

### **Citation index**

Stores citations or hyperlinks between documents to support citation analysis, a subject of Bibliometrics.

### **Ngram index**

Stores sequences of length of data to support other types of retrieval or text mining.

### **Document-term matrix**

Used in latent semantic analysis, stores the occurrences of words in documents in a two-dimensional sparse matrix.

## **(c) Challenges in parallelism**

- A major challenge in the design of search engines is the management of serial computing processes. There are many opportunities for race conditions and coherent faults.
- For example, a new document is added to the corpus and the index must be updated, but the index simultaneously needs to continue responding to search queries.
- This is a collision between two competing tasks. Consider that authors are producers of information, and a web crawler is the consumer of this information, grabbing the text and storing it in a cache (or corpus).

- The forward index is the consumer of the information produced by the corpus, and the inverted index is the consumer of information produced by the forward index. This is commonly referred to as a producer-consumer model.

#### **(d) Inverted indices:**

Many search engines incorporate an inverted index when evaluating a search query to quickly locate documents containing the words in a query and then rank these documents by relevance.

Because the inverted index stores a list of the documents containing each word, the search engine can use direct access to find the documents associated with each word in the query in order to retrieve the matching documents quickly.

#### **(e) Index merging**

- The inverted index is filled via a merge or rebuild. A rebuild is similar to a merge but first deletes the contents of the inverted index.
- The architecture may be designed to support incremental indexing, where a merge identifies the document or documents to be added or updated and then parses each document into words.
- For technical accuracy, a merge conflates newly indexed documents, typically residing in virtual memory, with the index cache residing on one or more computer hard drives.

### **4.4.2 Hashing**

Hashing is a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed.

#### **4.4.2.1 Static Hashing**

- A bucket is a unit of storage containing one or more records (a bucket is typically a disk block).
- In a hash file organization we obtain the bucket of a record directly from its search-key value using a hash function.
- Hash function  $h$  is a function from the set of all search-key values  $K$  to the set of all bucket addresses  $B$ .
- Hash function is used to locate records for access, insertion as well as deletion.

- Records with different search-key values may be mapped to the same bucket; thus entire bucket has to be searched sequentially to locate a record.

### Example of Hash File Organization:

- Hash file organization of account file, using branch-name as key
  - There are 10 buckets,
  - The binary representation of the  $i$ th character is assumed to be the integer  $i$ .
  - The hash function returns the sum of the binary representations of the characters modulo 10.

E.g.  $h(\text{Perryridge}) = 5$   $h(\text{Round Hill}) = 3$   $h(\text{Brighton}) = 3$

- Hash file organization of account file, using branch-name as key
  - Typical hash functions perform computation on the internal binary representation of the search-key.

Example, for a string search-key, the binary representations of all the characters in the string could be added and the sum modulo the number of buckets could be returned.

#### 4.4.2.2. Handling of Bucket Overflows

- Bucket overflow can occur because of Insufficient buckets

|          |       |            |     |
|----------|-------|------------|-----|
| bucket 0 |       |            |     |
| bucket 1 |       |            |     |
| bucket 2 |       |            |     |
| bucket 3 | A-217 | Brighton   | 750 |
|          | A-305 | Round Hill | 350 |
| bucket 4 | A-222 | Redwood    | 700 |
|          |       |            |     |
| bucket 5 | A-102 | Perryridge | 400 |
|          | A-201 | Perryridge | 900 |
|          | A-218 | Perryridge | 700 |
| bucket 6 |       |            |     |
| bucket 7 | A-215 | Mianus     | 700 |
|          |       |            |     |
| bucket 8 | A-101 | Downtown   | 500 |
|          | A-110 | Downtown   | 600 |
|          |       |            |     |
| bucket 9 |       |            |     |
|          |       |            |     |

Fig 4.5: Handling of Buckets overflow

- Skew in distribution of records.
- Multiple records have same search-key value
- Chosen hash function produces non-uniform distribution of key values
- Although the probability of bucket overflow can be reduced, it cannot be eliminated; it is handled by using overflow buckets.
- Overflow chaining – the overflow buckets of a given bucket are chained together in a linked list.
- The Above scheme is called closed hashing.
- An alternative, called open hashing, which does not use overflow buckets, is not suitable for database applications.

#### 4.4.2.3 Hash Indices

- Hashing can be used not only for file organization, but also for index-structure creation.
- A hash index organizes the search keys, with their associated record pointers, into a hash file structure.
- Strictly speaking, hash indices are always secondary indices
- If the file itself is organized using hashing, a separate primary hash index on it using the same search-key is unnecessary.
- However, we use the term hash index to refer to both secondary index structures and hash organized files.

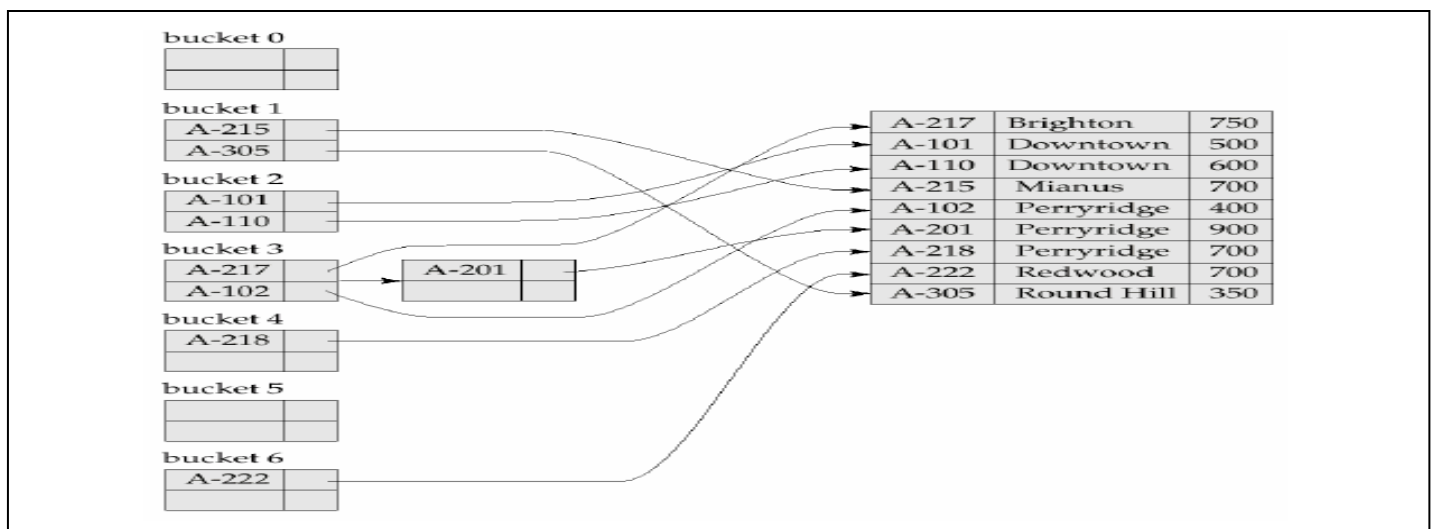


Fig 4.6: Example of Hash Index

---

### Deficiencies of Static Hashing

- In static hashing, function  $h$  maps search-key values to a fixed set of  $B$  of bucket addresses.
- Databases grow with time. If initial number of buckets is too small, performance will degrade due to too much overflows.
- If file size at some point in the future is anticipated and number of buckets allocated accordingly, significant amount of space will be wasted initially.
- If database shrinks, again space will be wasted.
- One option is periodic re-organization of the file with a new hash function, but it is very expensive.
- These problems can be avoided by using techniques that allow the number of buckets to be modified dynamically

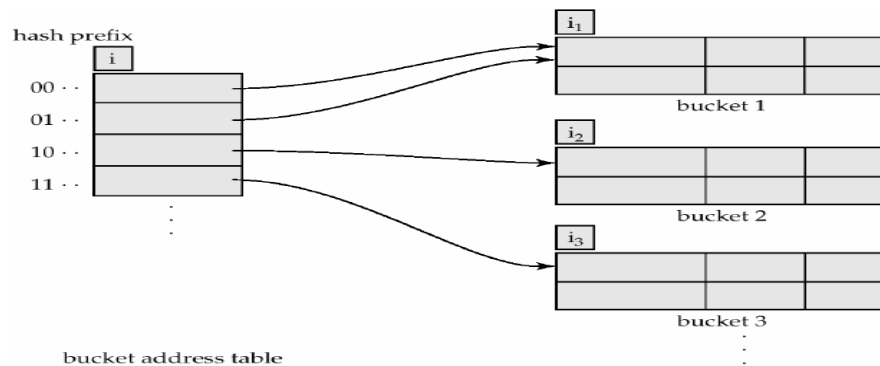
#### 4.4.2.4. Dynamic Hashing

The problem with static hashing is that it does not expand or shrink dynamically as the size of the database grows or shrinks. Dynamic hashing provides a mechanism in which data buckets are added and removed dynamically and on-demand. Dynamic hashing is also known as extended hashing.

#### Extendable hashing – one form of dynamic hashing

- Hash function generates values over a large range — typically  $b$ -bit integers, with  $b = 32$ .
- At any time use only a prefix of the hash function to index into a table of bucket addresses.
- Let the length of the prefix be  $i$  bits,  $0 \leq i \leq 32$ .
- Bucket address table size =  $2^i$ . Initially  $i = 0$ .
- Value of  $i$  grows and shrinks as the size of the database grows and shrinks.
- Multiple entries in the bucket address table may point to a bucket.
- Thus, actual number of buckets is  $< 2^i$ .
- The number of buckets also changes dynamically due to coalescing and splitting of buckets.

## General Extendable Hash Structure



**Fig 4.7: General Extendable Hash Structure**

In this structure,  $i_2 = i_3 = i$ , whereas  $i_1 = i - 1$

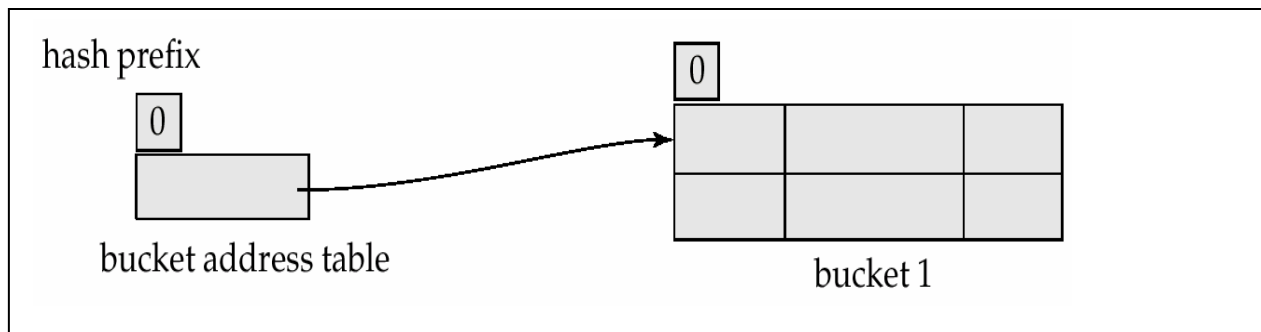
### Use of Extendable Hash Structure

- ✓ Each bucket  $j$  stores a value  $i_j$ ; all the entries that point to the same bucket have the same values on the first  $i_j$  bits.
- ✓ To locate the bucket containing search-key  $K_j$ :
- ✓ Compute  $h(K_j) = X$
- ✓ Use the first  $i$  high order bits of  $X$  as a displacement into bucket address table, and follow the pointer to appropriate bucket
- ✓ To insert a record with search-key value  $K_j$
- ✓ Follow same procedure as look-up and locate the bucket, say  $j$ .
- ✓ If there is room in the bucket  $j$  insert record in the bucket.
- ✓ Else the bucket must be split and insertion re-attempted.
- ✓ Overflow buckets used instead in some cases.

### Updates in Extendable Hash Structure

- ✓ To split a bucket  $j$  when inserting record with search-key value  $K_j$ :
- ✓ If  $i > i_j$  (more than one pointer to bucket  $j$ )
- ✓ Allocate a new bucket  $z$ , and set  $i_j$  and  $i_z$  to the old  $i_j + 1$ .
- ✓ Make the second half of the bucket address table entries pointing to  $j$  to point to  $z$
- ✓ Remove and reinsert each record in bucket  $j$ .
- ✓ Recompute new bucket for  $K_j$  and insert record in the bucket (further splitting is required if the bucket is still full)

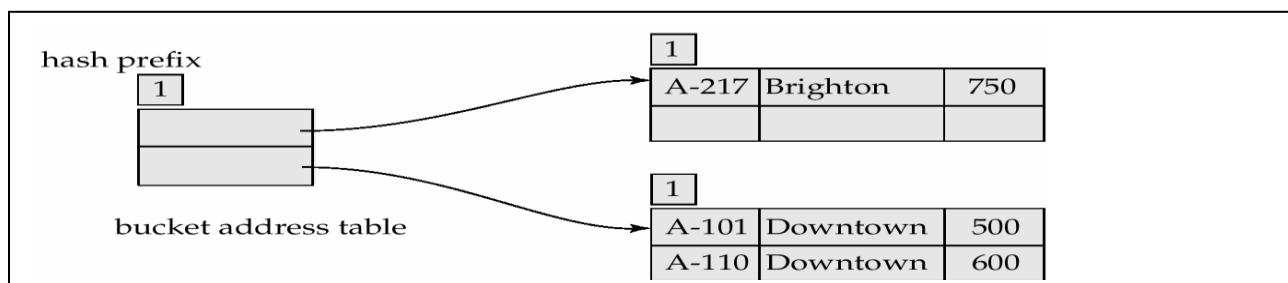
- ✓ If  $i = ij$  (only one pointer to bucket  $j$ )
- ✓ increment  $i$  and double the size of the bucket address table.
- ✓ Replace each entry in the table by two entries that point to the same bucket.
- ✓ Recompute new bucket address table entry for  $K_j$  Now  $i > ij$  so use the first case above.
- ✓ When inserting a value, if the bucket is full after several splits (that is,  $i$  reaches some limit  $b$ ) create an overflow bucket instead of splitting bucket entry table further.
- ✓ To delete a key value,
- ✓ Locate it in its bucket and remove it.
- ✓ Decreasing bucket address table size is also possible.

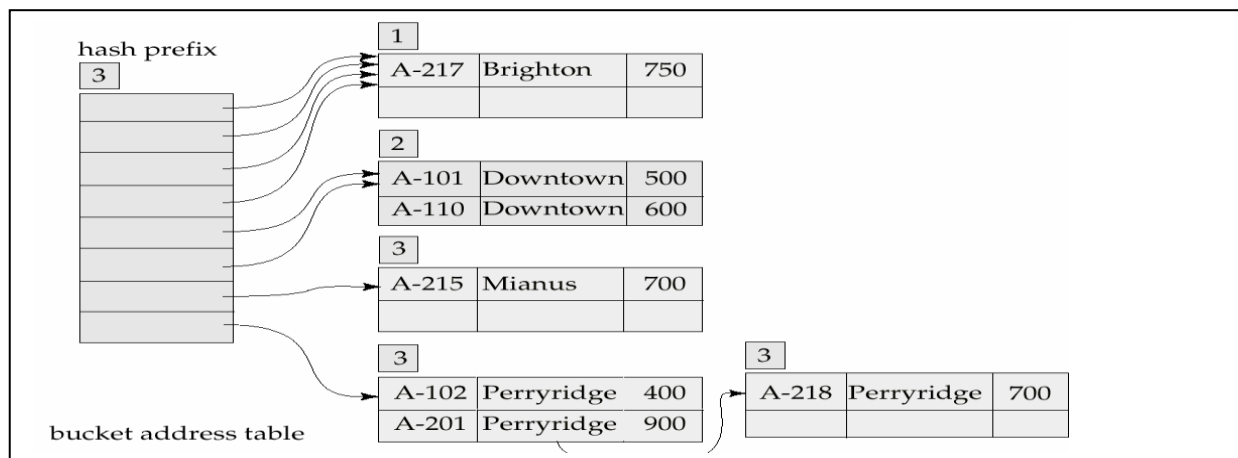
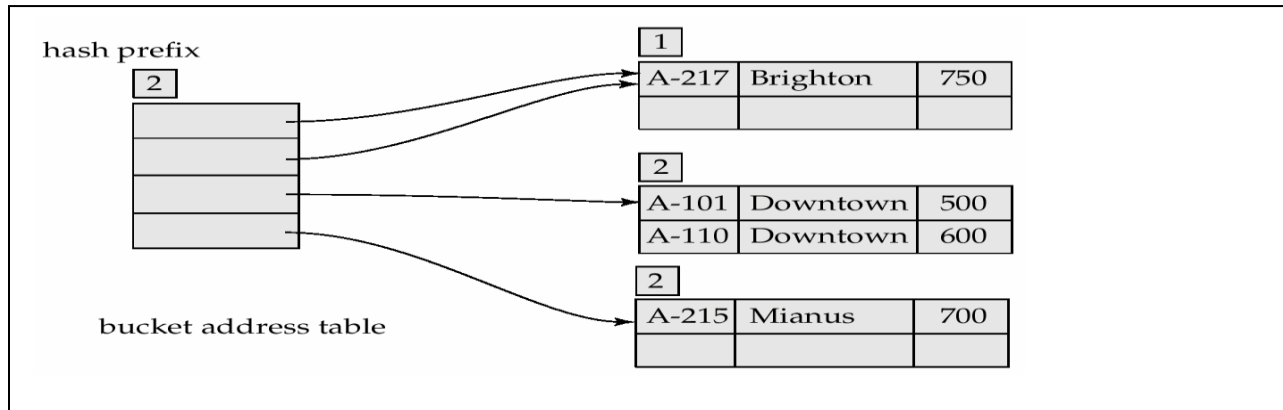


**Fig 4.8: Use of Extendable Hash Structure**

Initial Hash structure, bucket size = 2

- Hash structure after insertion of one Brighton and two Downtown Records
- Hash structure after insertion of Mianus record





- Hash structure after insertion of three Perryridge records
- Hash structure after insertion of Redwood and Round Hill records

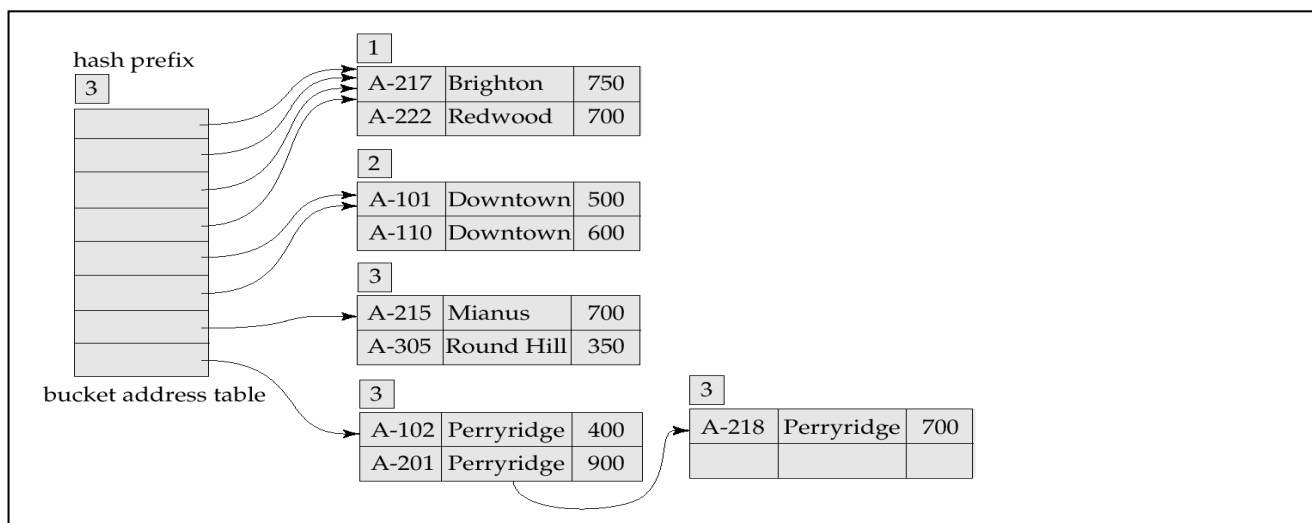


Fig 4.7: Hash table structure

#### 4.4.2.5. Extendable Hashing vs. Other Schemes

- Benefits of extendable hashing:
  - ✓ Hash performance does not degrade with growth of file
  - ✓ Minimal space overhead
- Disadvantages of extendable hashing
  - Extra level of indirection to find desired record
  - Bucket address table may itself become very big (larger than memory)
- Need a tree structure to locate desired record in the structure
  - Changing size of bucket address table is an expensive operation
- Linear hashing is an alternative mechanism which avoids these disadvantages at the possible cost of more bucket overflows.

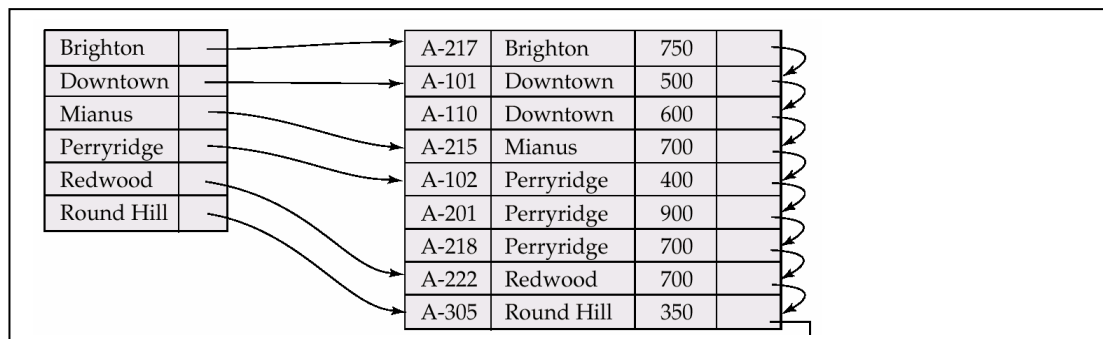
#### 4.5 ORDERED INDICES

- Indexing techniques evaluated on basis of:
  - ✓ In an ordered index, index entries are stored, sorted on the search key value. E.g., author catalog in library.
  - ✓ Primary index: in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
  - ✓ Also called clustering index
  - ✓ The search key of a primary index is usually but not necessarily the primary key.
  - ✓ Secondary index: an index whose search key specifies an order different from the sequential order of the file. Also called non-clustering index.

Index-sequential file: ordered sequential file with a primary index

##### 4.5.1 Dense Index Files

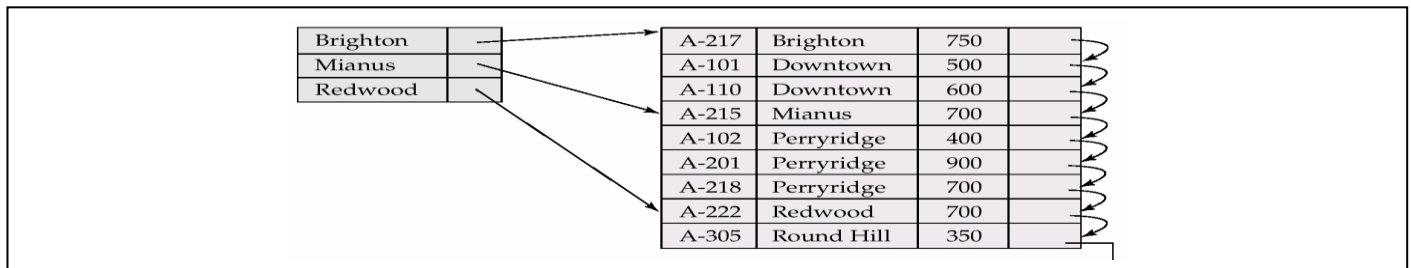
Dense index is the Index record appears for every search key value in the file.



**Fig 4.7: Ordered indices**

### 4.5.2 Sparse Index Files

- Sparse Index: contains index records for only some search-key values.
- Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value K we:
  - Find index record with largest search-key value  $< K$
  - Search file sequentially starting at the record to which the index record points.
- Less space and less maintenance overhead for insertions and deletions.
- Generally slower than dense index for locating records.
- Good tradeoff: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.
- Example of Sparse Index Files



**Fig 4.12: Sparse Index Files**

### 4.5.3 Multilevel Index

- If primary index does not fit in memory, access becomes expensive.
- To reduce number of disk accesses to index records, treat primary index kept on disk as a sequential file and construct a sparse index on it.
  - ✓ outer index – a sparse index of primary index
  - ✓ inner index – the primary index file
- If even the outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.

### Index Update: Deletion

- If deleted record was the only record in the file with its particular search-key value, the search-key is deleted from the index also.
- Single-level index deletion:
  - ✓ Dense indices – deletion of search-key is similar to file record deletion.

- ✓ Sparse indices – if an entry for the search key exists in the index, it is deleted by replacing the entry in the index with the next search-key value in the file (in search-key order). The next search-key value already has an index entry, the entry is deleted instead of being replaced.

### Index Update: Insertion

- Single-level index insertion:
- Perform a lookup using the search-key value appearing in the record to be inserted.
- Dense indices – if the search-key value does not appear in the index, insert it.
- Sparse indices – if index stores an entry for each block of the file, no change needs to be to the index unless a new block is created. In this case, the first search-key value appearing in the new block is inserted into the index.
- Multilevel insertion (as well as deletion) algorithms are simple extensions of the single-level algorithms.

### 4.5.4 Secondary Indices

- Frequently, one wants to find all the records whose values in a certain field (which is not the search-key of the primary index) satisfy some condition.

Example 1: In the account database stored sequentially by account number, we may want to find all accounts in a particular branch.

Example 2: as above, but where we want to find all accounts with a specified balance or range of balances

- We can have a secondary index with an index record for each search-key value; index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary Index on balance field of account

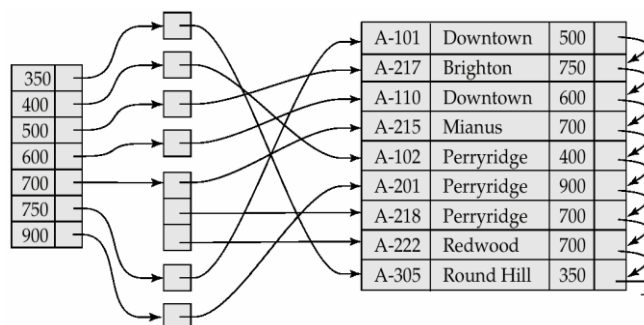


Fig 4.9: Secondary index

---

**Primary and Secondary Indices:**

- Secondary indices have to be dense.
- Indices offer substantial benefits when searching for records.
- When a file is modified, every index on the file must be updated, Updating indices imposes overhead on database modification.
- Sequential scan using primary index is efficient, but a sequential scan using a secondary index is expensive.
- each record access may fetch a new block from disk

**4.5.5 Bitmap Indices**

- Bitmap indices are a special type of index designed for efficient querying on multiple keys.
- Records in a relation are assumed to be numbered sequentially from, say, 0
- Given a number n it must be easy to retrieve record n

**Records are of fixed size:**

- Applicable on attributes that take on a relatively small number of distinct values
  - E.g. gender, country, state,
  - E.g. income-level (income broken up into a small number of levels such as 0-9999, 10000-19999, 20000-50000, 50000- infinity)
- A bitmap is simply an array of bits.
- In its simplest form a bitmap index on an attribute has a bitmap for each value of the attribute.
  - ✓ Bitmap has as many bits as records.
  - ✓ In a bitmap for value v, the bit for a record is 1 if the record has the value v for the attribute, and is 0 otherwise.
- Bitmap indices are useful for queries on multiple attributes
  - ✓ Not particularly useful for single attribute queries
- Queries are answered using bitmap operations
  - ✓ Intersection (and)
  - ✓ Union (or)
  - ✓ Complementation (not)
- Each operation takes two bitmaps of the same size and applies the operation on corresponding bits to get the result bitmap

- ✓ Can then retrieve required tuples.
- ✓ Counting number of matching tuples is even faster
- Bitmap indices generally very small compared with relation size.
  - ✓ If number of distinct attribute values is 8, bitmap is only 1% of relation size
- Deletion needs to be handled properly
  - ✓ Existence bitmap to note if there is a valid record at a record location
  - ✓ Needed for complementation

#### 4.6. B+-TREE INDEX FILES

- B+-tree indices are an alternative to indexed-sequential files.
- Disadvantage of indexed-sequential files: performance degrades as file grows, since many overflow blocks get created. Periodic reorganization of entire file is required.
- Advantage of B+-tree index files: automatically reorganizes itself with small, local, changes, in the face of insertions and deletions. Reorganization of entire file is not required to maintain performance.
- Disadvantage of B+-trees: extra insertion and deletion overhead, space overhead.
- Advantages of B+-trees outweigh disadvantages, and they are used extensively.
- A B+-tree is a rooted tree satisfying the following properties:
  - ✓ All paths from root to leaf are of the same length
  - ✓ Each node that is not a root or a leaf has between  $\lceil n/2 \rceil$  and  $n$  children.
  - ✓ A leaf node has between  $\lceil (n-1)/2 \rceil$  and  $n-1$  values
  - ✓ Special cases:
    - If the root is not a leaf, it has at least 2 children.
    - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and  $(n-1)$  values.
  - ✓ Typical node



**Fig 4.10: B+ tree index files typical node**

- $K_i$  are the search-key values
  - $P_i$  are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \dots < K_{n-1}.$$

#### 4.6.1 Leaf Nodes in B+-Trees

##### Properties of a leaf node:

- For  $i = 1, 2, \dots, n-1$ , pointer  $P_i$  either points to a file record with search-key value  $K_i$ , or to a bucket of pointers to file records, each record having search-key value  $K_i$ . Only need bucket structure if search-key does not form a primary key.
- If  $L_i, L_j$  are leaf nodes and  $i < j$ ,  $L_i$ 's search-key values are less than  $L_j$ 's search-key values.

$P_n$  points to next leaf node in search-key order

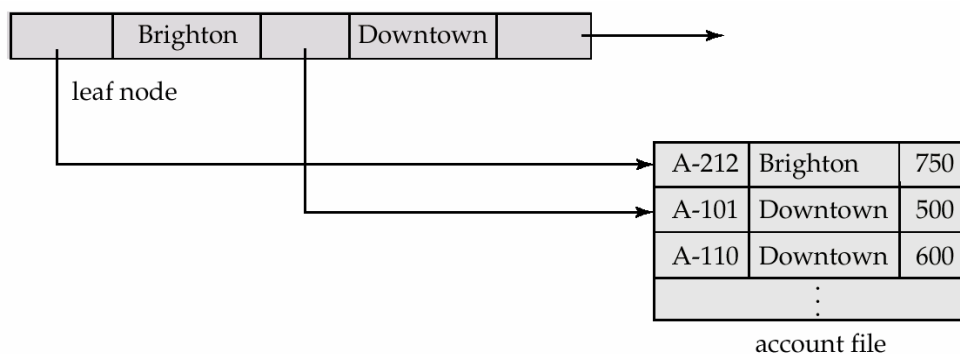


Fig 4.11: Leaf node in B+ tree

#### 4.6.2 Non-Leaf Nodes in B+-Trees:

- Non leaf nodes form a multi-level sparse index on the leaf nodes. For a non-leaf node with  $m$  pointers:

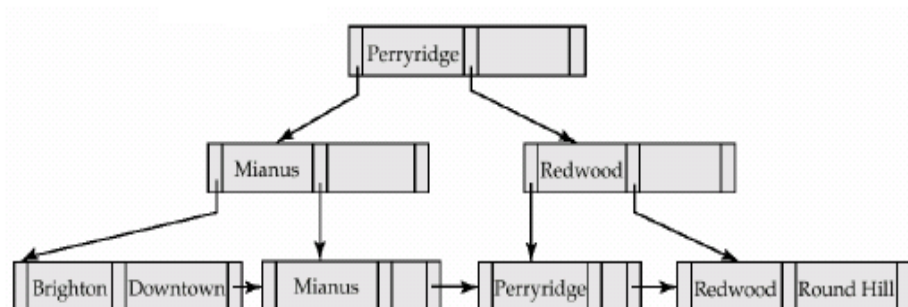
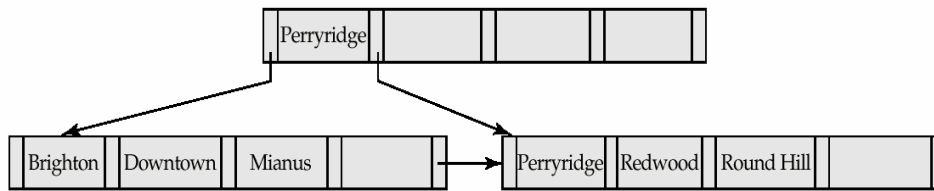


Fig 4.12: Non Leaf node in B+ tree

B+-tree for account file ( $n = 3$ )



**Fig 4.13: B+-tree for account file**

- Leaf nodes must have between 2 and 4 values ( $(n-1)/2$  and  $n-1$ , with  $n = 5$ ).
- Non-leaf nodes other than root must have between 3 and 5 children ( $(n/2)$  and  $n$  with  $n=5$ ).
- Root must have at least 2 children.

#### 4.6.3 Observations about B+-trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B+-tree form a hierarchy of sparse indices.
- The B+-tree contains a relatively small number of levels (logarithmic in the size of the main file), thus searches can be conducted efficiently.
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time.

#### Queries on B+-Trees

- Find all records with a search-key value of  $k$ .
- Start with the root node
  - ✓ Examine the node for the smallest search-key value  $> k$ .
  - ✓ If such a value exists, assume it is  $K_j$ . Then follow  $P_i$  to the child node.
  - ✓ Otherwise  $k = K_{m-1}$ , where there are  $m$  pointers in the node. Then follow  $P_m$  to the child node.
- If the node reached by following the pointer above is not a leaf node, repeat the above procedure on the node, and follow the corresponding pointer.
- Eventually reach a leaf node. If for some  $i$ , key  $K_i = k$  follow pointer  $P_i$  to the desired record or bucket. Else no record with search-key value  $k$  exists.
- In processing a query, a path is traversed in the tree from the root to some leaf node.
- If there are  $K$  search-key values in the file, the path is no longer than

$$\log_{\lceil n/2 \rceil}(K).$$

- A node is generally the same size as a disk block, typically 4 kilobytes, and n is typically around 100 (40 bytes per index entry).
- With 1 million search key values and n = 100, at most  $\log_{50}(1,000,000) = 4$  nodes are accessed in a lookup.
- Contrast this with a balanced binary tree with 1 million search key values — around 20 nodes are accessed in a lookup.
  - ✓ The above difference is significant since every node access may need a disk I/O, costing around 20 milliseconds!

### Updates on B+-Trees: Insertion

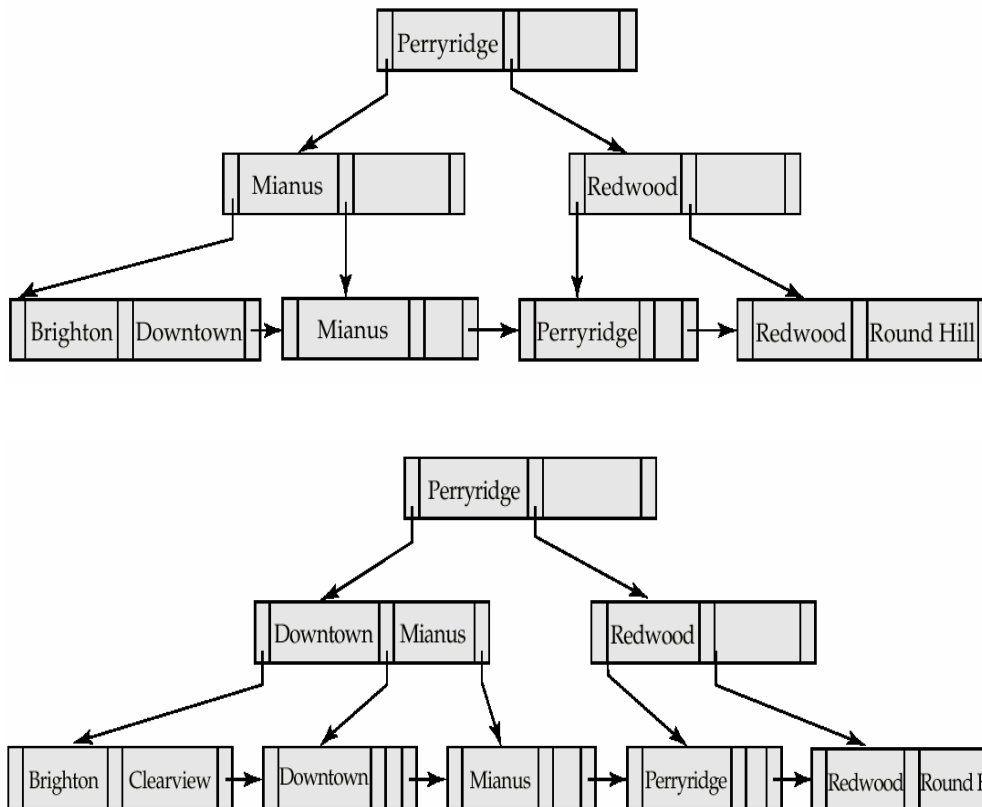
- Find the leaf node in which the search-key value would appear
- If the search-key value is already there in the leaf node, record is added to file and if necessary a pointer is inserted into the bucket.
- If the search-key value is not there, then add the record to the main file and create a bucket if necessary. Then:
- If there is room in the leaf node, insert (key-value, pointer) pair in the leaf node
- Otherwise, split the node (along with the new (key-value, pointer) entry) .

### Splitting a node:

- Take the n(search-key value, pointer) pairs (including the one being inserted) in sorted order. Place the first n/2 in the original node and the rest in a new node.
- let the new node be p, and let k be the least key value in p. Insert (k,p) in the parent of the node being split. If the parent is full, split it and propagate the split further up.
- The splitting of nodes proceeds upwards till a node that is not full is found. In the worst case the root node may be split increasing the height of the tree by 1.



Result of splitting node containing Brighton and Downtown on inserting Clear view

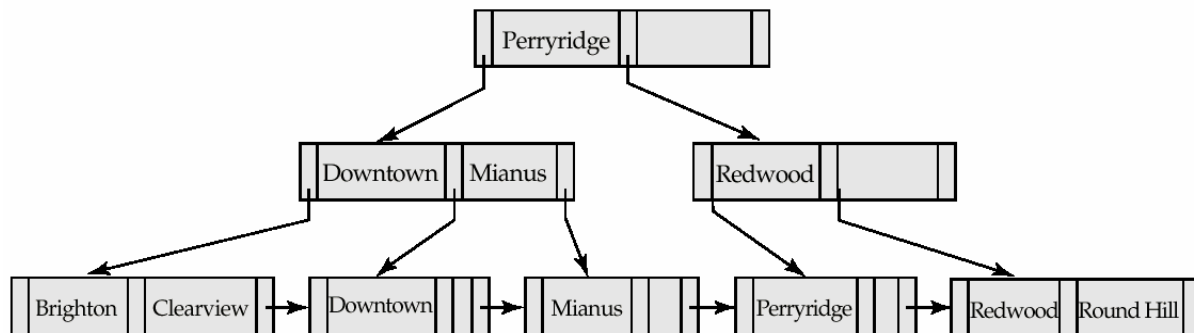


**Fig 4.14: B+-Tree before and after insertion of "Clear view"**

### Updates on B+-Trees: Deletion

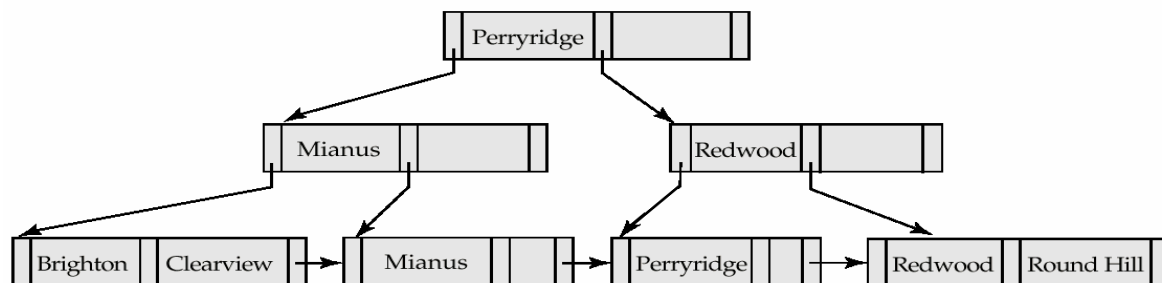
- Find the record to be deleted, and remove it from the main file and from the bucket
- Remove (search-key value, pointer) from the leaf node if there is no bucket or if the bucket has become empty.
- If the node has too few entries due to the removal, and the entries in the node and a sibling fit into a single node, then
  - Insert all the search-key values in the two nodes into a single node (the one on the left), and delete the other node.
  - Delete the pair  $(K_{i-1}, P_i)$ , where  $P_i$  is the pointer to the deleted node, from its parent, recursively using the above procedure.
- Otherwise, if the node has too few entries due to the removal, and the entries in the node and a sibling fit into a single node, then
  - Redistribute the pointers between the node and a sibling such that both have more than the minimum number of entries.

- Update the corresponding search-key value in the parent of the node.

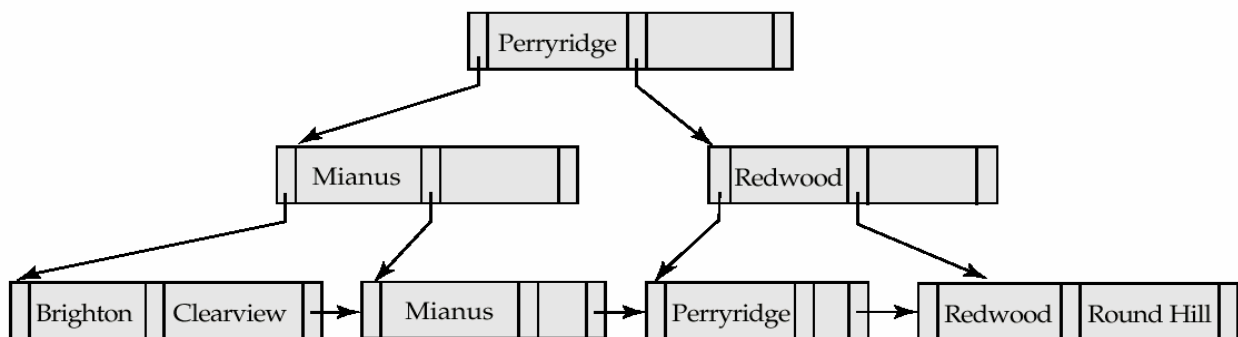


**Fig 4.15: B+-Tree Deletion**

Before and after deleting “Downtown”



- The removal of the leaf node containing “Downtown” did not result in its parent having too little pointers. So the cascaded deletions stopped with the deleted leaf node’s parent



**Fig 4.16: Deletion of Perryridge**

- The node deletions may cascade upwards till a node which has  $\lceil n/2 \rceil$  or more pointers is found. If the root node has only one pointer after deletion, it is deleted and the sole child becomes the root.

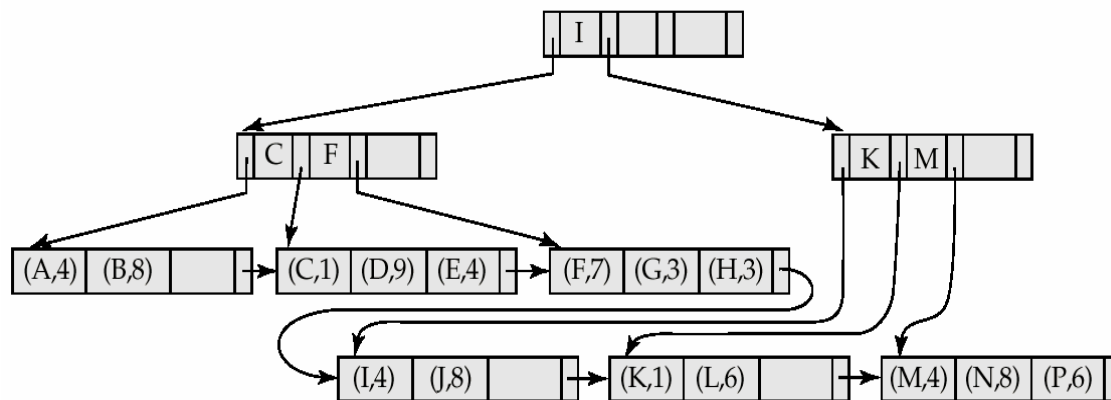
Deletion of “Perryridge” from result of previous example

- Node with “Perryridge” becomes under full (actually empty, in this special case) and is merged with its sibling.
- As a result “Perryridge” node’s parent became under full, and was merged with its sibling (and an entry was deleted from their parent).
- Root node then had only one child, and was deleted and its child became the new root node.

#### 4.6.4 B+-Tree File Organization

- Index file degradation problem is solved by using B+-Tree indices. Data file degradation problem is solved by using B+-Tree File Organization.
- The leaf nodes in a B+-tree file organization store records, instead of pointers.
- Since records are larger than pointers, the maximum number of records that can be stored in a leaf node is less than the number of pointers in a non leaf node.
- Leaf nodes are still required to be half full.

Insertion and deletion are handled in the same way as insertion and deletion of entries in a B+-tree index



**Fig 4.17: B+-tree File Organization**

- Good space utilization is important since records use more space than pointers.
- To improve space utilization, involve more sibling nodes in redistribution during splits and merges.

3. Involving 2 siblings in redistribution (to avoid split / merge where possible) results in each node having at least entries

#### 4.7 B-TREE INDEX FILES

- ✓ Similar to B+-tree, but B-tree allows search-key values to appear only once; eliminates redundant storage of search keys.
- ✓ Search keys in nonleaf nodes appear nowhere else in the B-tree; an additional pointer field for each search key in a nonleaf node must be included.
- ✓ Generalized B-tree leaf node



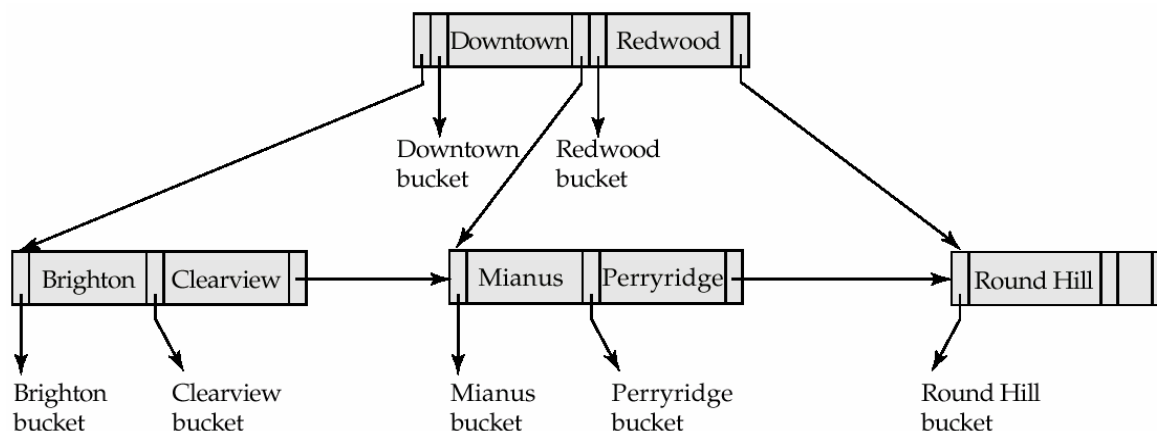
(a)



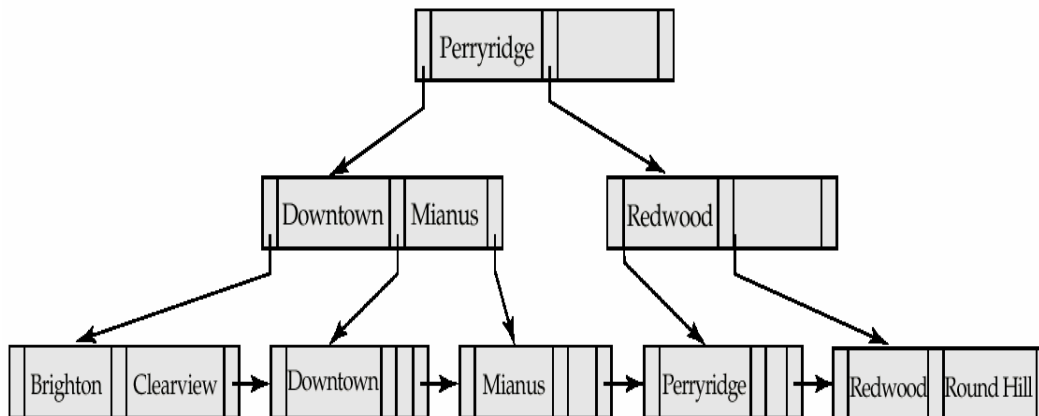
(b)

**Fig 4.18: B-tree leaf node**

#### B-Tree Index File Example



**Fig 4.19: B-tree (above) and B+-tree (below) on same data**



#### 4.7.1 Advantages of B-Tree indices:

- May use less tree nodes than a corresponding B+-Tree.
- Sometimes possible to find search-key value before reaching leaf node.

#### 4.7.2 Disadvantages of B-Tree indices:

- Only small fraction of all search-key values are found early
  - Non-leaf nodes are larger, so fan-out is reduced. Thus B-Trees typically have greater depth than corresponding B+-Tree
    - ✓ Insertion and deletion more complicated than in B+-Trees
    - ✓ Implementation is harder than B+-Trees.
- Typically, advantages of B-Trees do not outweigh disadvantages.

### 4.8 QUERY PROCESSING AND OPTIMIZATION

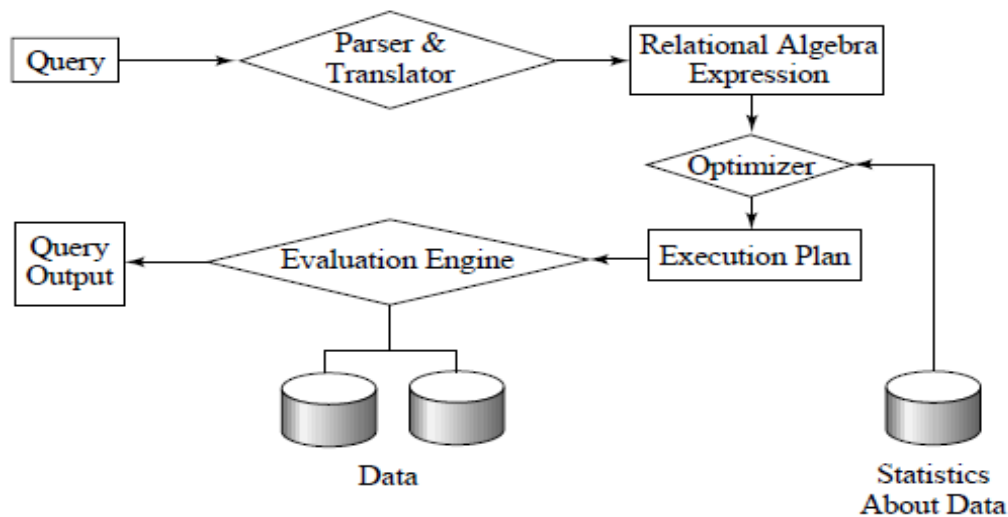
#### 4.8.1 Query Processing

The retrieval of information from a database according to a set of retrieval criteria, the database itself remaining unchanged.

In the context of a specific query language, the technique of translating the retrieval criteria specified using the language into more primitive database-access software.

##### 4.8.1.1 Basic steps in query processing

1. Parsing and translation
2. Optimization
3. Evaluation

**Fig4.20: Query Processing**

### 1. Parsing and translation

- Translate the query into its internal form. This is then translated into relational algebra.
- Parser checks syntax, verifies relations

### 2. Evaluation

- ✓ The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.

### 3. Optimization

- ✓ Finding the cheapest evaluation plan for a query.
- ✓ Given relational algebra expression may have many equivalent expressions
- ✓ Any relational-algebra expression can be evaluated in many ways. Annotated expression specifying detailed evaluation strategy is called an evaluation-plan
- ✓ Amongst all equivalent expressions, try to choose the one with cheapest possible evaluation-plan. Cost estimate of a plan based on statistical information in the DBMS catalog

#### 4.8.1.2 Selection operation

- **File scan** – search algorithms that locate and retrieve records that fulfill a selection condition.
- **Algorithm A1** (linear search). Scan each file block and test all records to see whether they satisfy the selection condition.
  - Cost estimate (number of disk blocks scanned)  $EA1 = br$
  - If selection is on a key attribute,  $EA1 = (br / 2)$  (stop on finding record)
  - Linear search can be applied regardless of
    - ✓ selection condition, or
    - ✓ ordering of records in the file, or
    - ✓ availability of indices
- **A2** (binary search). Applicable if selection is an equality comparison on the attribute on which file is ordered.
  - Assume that the blocks of a relation are stored contiguously
  - Cost estimate (number of disk blocks to be scanned):
  - Equality condition on a key attribute:  $SC(A, r) = 1$ ; estimate reduces to  $EA2 = \lceil \log_2(br) \rceil$
- **Index scan** – search algorithms that use an index; condition is on search-key of index.
- **A3** (primary index on candidate key, equality). Retrieve a single record that satisfies the corresponding equality condition.  $EA3 = HT_i + 1$
- **A4** (primary index on nonkey, equality) Retrieve multiple records. Let the search-key attribute be A
- **A5** (equality on search-key of secondary index).
- **A6** (primary index, comparison). The cost estimate is:
 
$$EA6 = HT_i + c/fr$$

where c is the estimated number of tuples satisfying the condition. In absence of statistical information c is assumed to be  $nr / 2$ .
- **A7** (secondary index, comparison). The cost estimate is:
 
$$EA7 = HT_i + (LB_i - c/nr) + c$$

---

where  $c$  is defined as before. (Linear file scan may be cheaper if  $c$  is large!)

- **A8** (conjunctive selection using one index). Select a combination of  $\theta_i$  and algorithms A1 through A7 that results in the least cost for  $\theta_i(r)$ . Test other conditions in memory buffer.
- **A9** (conjunctive selection using multiple-key index). Use appropriate composite (multiple-key) index if available.
- **A10** (conjunctive selection by intersection of identifiers). Requires indices with record pointers. Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers. Then read file. If some conditions did not have appropriate indices, apply test in memory.
- **A11** (disjunctive selection by union of identifiers). Applicable if all conditions have available indices. Otherwise use linear scan.

#### 4.8.2 Query Optimization

Query optimization is a function of many relational database management systems. The query optimizer attempts to determine the most efficient way to execute a given query by considering the possible query plans.

The query optimizer (called simply the optimizer) is built-in database software that determines the most efficient method for a SQL statement to access requested data.

- Purpose of the Query Optimizer
- Cost-Based Optimization
- Execution Plans

##### 1. Purpose of the Query Optimizer

The optimizer attempts to generate the best execution plan for a SQL statement. The best execution plan is defined as the plan with the lowest cost among all considered candidate plans.

##### 2 Cost-Based Optimization

Query optimization is the overall process of choosing the most efficient means of executing a SQL statement. SQL is a nonprocedural language, so the optimizer is free to merge, reorganize, and process in any order.

The database optimizes each SQL statement based on statistics collected about the accessed data. When generating execution plans, the optimizer considers different access paths and joins methods. Factors considered by the optimizer include:

- System resources, which includes I/O, CPU, and memory
- Number of rows returned
- Size of the initial data sets

### 3. Execution Plans

An execution plan describes a recommended method of execution for a SQL statement. The plan shows the combination of the steps Oracle Database uses to execute a SQL statement.

#### 4.8.2.1 About Optimizer Components

The optimizer contains three main components, which are shown in below diagram

A set of query blocks represents a parsed query, which is the input to the optimizer. The optimizer performs the following operations:

##### Query transformer

The optimizer determines whether it is helpful to change the form of the query so that the optimizer can generate a better execution plan.

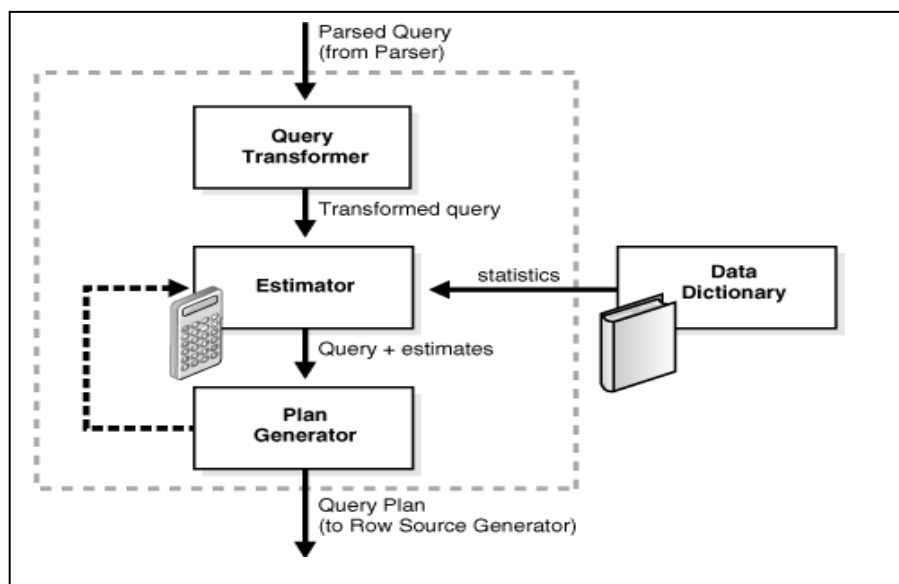


Fig 4.21: Optimizer Components

---

- **Estimator**

The optimizer estimates the cost of each plan based on statistics in the data dictionary.

- **Plan Generator**

The optimizer compares the costs of plans and chooses the lowest-cost plan, known as the execution plan, to pass to the row source generator

## **4.9 ALGORITHMS FOR SELECT AND JOIN OPERATIONS**

### **4.9.1 Implementing the SELECT Operation**

Five operations for demonstration:

- (OP1):  $\sigma_{SSN=01234567890}$  (EMPLOY EE)
- (OP2):  $\sigma_{DNUMBER>5}$  (DEPARTMENT)
- (OP3):  $\sigma_{DNO=5}$  (EMPLOY EE)
- (OP4):  $\sigma_{DNO=5 \text{ and } SALARY > 30000 \text{ and } SEX=0F0}$  (EMPLOY EE)
- (OP5):  $\sigma_{ESSN=01234567890 \text{ and } P NO=10}$  (WORKS ON)

#### **Search methods for simple selection:**

- S1. Linear search (brute force)
- S2. Binary search: If the selection condition involves an equality comparison on a key attribute on which the file is ordered. (ex. OP1)
- S3. Using a primary index: If the selection condition involves an equality comparison on a key attribute with a primary index. (ex. OP1)
- S4. Using a primary index to retrieve multiple records: If the comparison condition is  $>$ ,  $\leq$ ,  $<$ ,  $\geq$  on a key field with a primary index – for example,  $DNUMBER > 5$  in OP2 – use the index to find the record satisfying the equality condition ( $DNUMBER = 5$ ), then retrieve all subsequent (preceding) records in the (ordered) file.
- S5. Using a clustering index to retrieve multiple records: If the selection condition involves an equality comparison on a non-key attribute with a clustering index – for example,  $DNO = 5$  in OP3 – use the index to retrieve all the records satisfying the condition.
- S6. Using a secondary (B+-Tree) index on an equality comparison: Retrieve one record if the indexing field is a key, or multiple records if the indexing field is not a key. This method can also be used for comparison involved  $>$ ,  $\leq$ ,  $<$ ,  $\geq$ .

- 
- Search methods for complex (conjunctive) selection:
  - S7. Conjunctive selection using an individual index: If an attribute involved in any single simple condition in the conjunctive condition has an access path that permits the use of one of the methods S2 to S6 to retrieve the records and then check whether each retrieved record satisfies the remaining conditions in the conjunctive condition.
  - S8. Conjunctive selection using a composite index: If two or more attributes are involved in equality conditions and a composite index exists on the combined fields, we can use the index directly. For example, OP5 can use this method if there is a composite index (ESSN, PNO) for WORKS ON file.
  - S9. Conjunctive selection by intersection of record pointers: If secondary indexes are available on more than one of the fields in the conjunctive condition, and if the indexes include record pointers (rather than block pointers). The intersection of these sets of record pointers would satisfy the conjunctive condition, which are then used to retrieve those records directly. If only some of the simple conditions have secondary indexes, each retrieved record is further tested for the remaining conditions

#### 4.9.2 Implementing the JOIN Operation

Two operations for demonstration:

1. OP6): EMPLOY EE ./ DNO=DNUMBER DEPARTMENT
2. (OP7): DEPARTMENT ./ MGRSSN=SSN EMPLOY EE

Methods for implementing JOINS:

The algorithms we consider are for join operation of the form

$$R ./ A=B S$$

- J1. Nested-loop join (brute force): For each record  $t$  in  $R$  (outer loop), retrieve every record  $s$  from  $S$  and test whether the two records satisfy the join condition  $t[A] = s[B]$ .
- J2. Single-loop join (using an access structure to retrieve the matching records): If an index exists for one of the two attributes – say,  $B$  of  $S$  – retrieve each record  $t$  in  $R$ , one at a time (single loop), and then use the access structure to retrieve directly all matching records  $s$  from  $S$  that satisfy  $s[B] = t[A]$ .
- J3. Sort-merge join: First to sort  $R$  and  $S$  based on the corresponding attributes  $A$  and  $B$  using external sorting. Second to merge sorted  $R$  and  $S$  by retrieving the matching

records  $t$  and  $s$  that satisfy the join condition  $t[A] = s[B]$ . Note that if there are secondary indexes for  $R$  and  $S$  based on attributes  $A$  and  $B$ , then we can merge these two secondary indexes instead of sorting and merging the data files  $R$  and  $S$ .

➤ **J4. Hash-join:**

- **Partitioning phase:** a single pass through the file with fewer records (say,  $R$ ) hashes its records (using  $A$  as the hash key) to the hash file buckets
- **Probing phase:** a single pass through the other file ( $S$ ) then hashes each of its records to probe the appropriate bucket.

#### 4.10 HEURISTICS AND COST ESTIMATES IN QUERY OPTIMIZATION.

##### 4.10.1 Heuristics

- **Process for heuristics optimization**

The parser of a high-level query generates an initial internal representation;

Apply heuristics rules to optimize the internal representation.

A query execution plan is generated to execute groups of operations based on the access paths available on the files involved in the query.

➤ **The main heuristic is to apply first the operations that reduce the size of intermediate results.**

**Example:** Apply SELECT and PROJECT operations before applying the JOIN or other binary operations.

- ✓ **Query tree:** a tree data structure that corresponds to a relational algebra expression. It represents the input relations of the query as leaf nodes of the tree, and represents the relational algebra operations as internal nodes.

- ✓ **Query graph:** a graph data structure that corresponds to a relational calculus expression. It does **not** indicate an order on which operations to perform first. There is only a **single** graph corresponding to each query.

**Example:** For every project located in 'Stafford', retrieve the project number, the controlling department number and the department manager's last name, address and birthdate

##### 4.10.1.1 Relation algebra:

**p** PNUMBER, DNUM, LNAME, ADDRESS, BDATE (((s<sub>PLOCATION</sub>= 'STAFFORD')(PROJECT))

DNUM=DNUMBER (DEPARTMENT)) MGRSSN=SSN (EMPLOYEE))

### SQL query:

Q2: SELECT P.NUMBER, P.DNUM, E.LNAME, E.ADDRESS, E.BDATE

FROM

PROJECT AS P,

DEPARTMENT AS D,

EMPLOYEE AS E

WHERE

P.DNUM=D.DNUMBER AND D.MGRSSN=E.SSN AND

P.PLOCATION='STAFFORD';

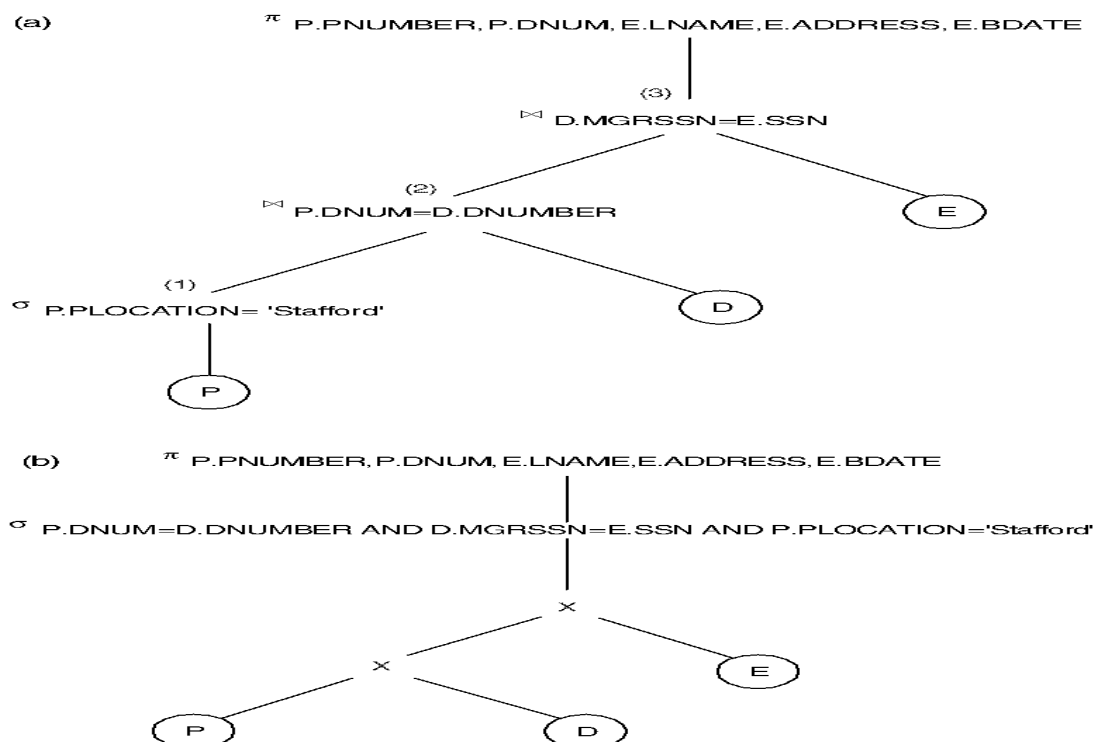


Fig 4.22: (a) Query tree corresponding to relational algebra expression Q2 (b) Initial query tree for sql Query Q2

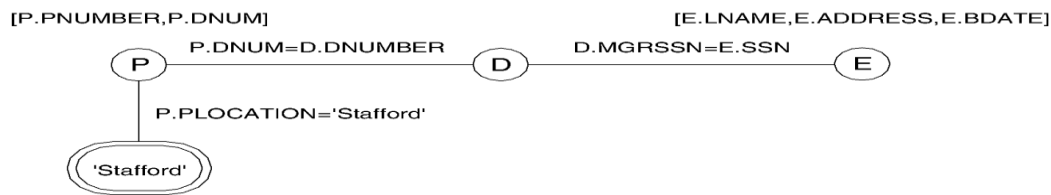


Fig c) Query graph for Q2

#### 4.10.1.2 General Transformation Rules for Relational Algebra Operations:

- Cascade of s: A conjunctive selection condition can be broken up into a cascade (sequence) of individual s operations:

$$S_{c1 \text{ AND } c2 \text{ AND } \dots \text{ AND } c_n}(R) = S_{c1}(S_{c2}(\dots(S_{c_n}(R))\dots))$$

- Commutativity of s: The s operation is commutative:

$$S_{c1}(S_{c2}(R)) = S_{c2}(S_{c1}(R))$$

- Cascade of p: In a cascade (sequence) of p operations, all but the last one can be ignored:

$$p_{List1}(p_{List2}(\dots(p_{Listn}(R))\dots)) = p_{List1}(R)$$

- ✓ Commuting s with p: If the selection condition c involves only the attributes A1, ..., An in the projection list, the two operations can be commuted:

$$p_{A1, A2, \dots, An}(S_c(R)) = S_c(p_{A1, A2, \dots, An}(R))$$

- Commutativity of (and x): The operation is commutative as is the x operation:

$$R \text{ } _c S = S \text{ } _c R; R \times S = S \times R$$

- Commuting s with (or x): If all the attributes in the selection condition c involve only the attributes of one of the relations being joined—say, R—the two operations can be commuted as follows :

$$S_c(R \text{ } S) = (S_c(R)) \text{ } S$$

Alternatively, if the selection condition c can be written as (c1 and c2), where condition c1 involves only the attributes of R and condition c2 involves only the attributes of S, the operations commute as follows:

$$S_c(R \text{ } S) = (S_{c1}(R)) \text{ } (S_{c2}(S))$$

- Commuting  $\pi$  with  $\sigma$  (or  $\bowtie$ ): Suppose that the projection list is  $L = \{A_1, \dots, A_n, B_1, \dots, B_m\}$ , where  $A_1, \dots, A_n$  are attributes of  $R$  and  $B_1, \dots, B_m$  are attributes of  $S$ . If the join condition  $c$  involves only attributes in  $L$ , the two operations can be commuted as follows:

$$\pi_L (R \bowtie_c S) = (\pi_{A_1, \dots, A_n} (R)) \bowtie_c (\pi_{B_1, \dots, B_m} (S))$$

If the join condition  $c$  contains additional attributes not in  $L$ , these must be added to the projection list, and a final  $\pi$  operation is needed.

- Commutativity of set operations: The set operations  $\cup$  and  $\cap$  are commutative but  $-$  is not.
- Associativity of  $\cup$ ,  $\cap$ , and  $\times$ : These four operations are individually associative; that is, if  $q$  stands for any one of these four operations (throughout the expression), we have  $(R \ q \ S) \ q \ T = R \ q \ (S \ q \ T)$
- Commuting  $\sigma$  with set operations: The  $\sigma$  operation commutes with  $\cup$ ,  $\cap$ , and  $-$ . If  $q$  stands for any one of these three operations, we have

$$\sigma_c (R \ q \ S) = (\sigma_c (R)) \ q \ (\sigma_c (S))$$

- ✓ The  $\pi$  operation commutes with  $\cup$ .

$$\pi_L (R \cup S) = (\pi_L (R)) \cup (\pi_L (S))$$

Converting a  $(\sigma, \times)$  sequence into: If the condition  $c$  of a  $\sigma$  that follows a  $\times$  corresponds to a join condition, convert the  $(\sigma, \times)$  sequence into a  $\bowtie$  as follows  $(\sigma_c (R \times S)) = (R \bowtie_c S)$

- ✓ Other transformations

#### 4.10.1.3 Outline of a Heuristic Algebraic Optimization Algorithm:

- Using rule 1, break up any select operations with conjunctive conditions into a cascade of select operations.
- Using rules 2, 4, 6, and 10 concerning the commutativity of select with other operations, move each select operation as far down the query tree as is permitted by the attributes involved in the select condition.
- Using rule 9 concerning associativity of binary operations, rearrange the leaf nodes of the tree so that the leaf node relations with the most restrictive select operations are executed first in the query tree representation.

- Using Rule 12, combine a Cartesian product operation with a subsequent select operation in the tree into a join operation.
- Using rules 3, 4, 7, and 11 concerning the cascading of project and the commuting of project with other operations, break down and move lists of projection attributes down the tree as far as possible by creating new project operations as needed.

Identify subtrees that represent groups of operations that can be executed by a single algorithm

#### 4.10.1.4 Summary of Heuristics for Algebraic Optimization:

- The main heuristic is to apply first the operations that reduce the size of intermediate results.
- Perform select operations as early as possible to reduce the number of tuples and perform project operations as early as possible to reduce the number of attributes. (This is done by moving select and project operations as far down the tree as possible)
- The select and join operations that are most restrictive should be executed before other similar operations. (This is done by reordering the leaf nodes of the tree among themselves and adjusting the rest of the tree appropriately.)

#### 4.10.2 Cost Estimates

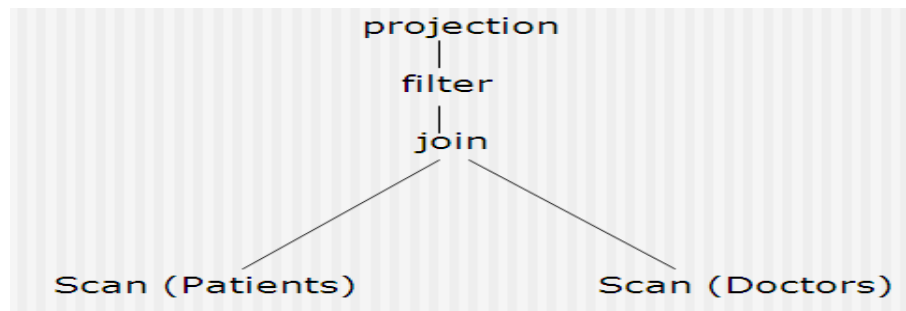
- One of the hardest problems in query optimization is to accurately estimate the costs of alternative query plans.
- Optimizers cost query plans using a mathematical model of query execution costs that relies heavily on estimates of the cardinality, or number of tuples, flowing through each edge in a query plan.
- Cardinality estimation in turn depends on estimates of the selection factor of predicates in the query.
- Traditionally, database systems estimate selectivity's through fairly detailed statistics on the distribution of values in each column, such as histograms.
- This technique works well for estimation of selectivity's of individual predicates.

##### 4.10.2.1 Algebraic Expressions

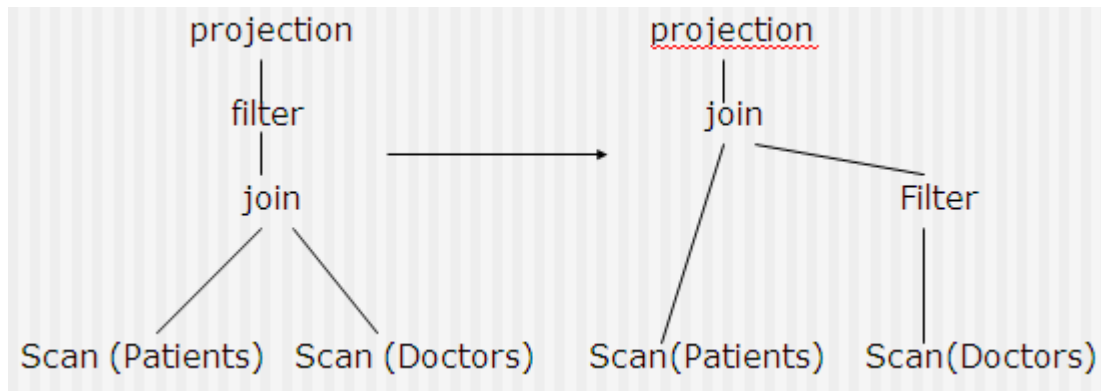
##### Cost-based query Optimization:

```
SELECT p.pname, d.dname FROM Patients p, Doctors d WHERE p.doctor =
```

d.dname AND d.dgender = 'M'



### Cost-based query Optimization: Transformation



### Cost-based query Optimization: Plan selection based on costs

